UNIVERSIDADE FEDERAL DO CEARÁ

CENTRO DE TECNOLOGIA

DEPARTAMENTO DE ENGENHARIA ELÉTRICA

CURSO DE GRADUAÇÃO EM ENGENHARIA ELÉTRICA

PEDRO FLORENCIO DE ALMEIDA NETO

COMPARAÇÃO ENTRE VISION TRANSFORMERS E U-NET COM MOBILENETV2 NA SEGMENTAÇÃO DE EXSUDATOS LIPÍDICOS EM IMAGENS DE RETINOGRAFIA

FORTALEZA

2026

PEDRO FLORENCIO DE ALMEIDA NETO

COMPARAÇÃO ENTRE VISION TRANSFORMERS E U-NET COM MOBILENETV2 NA SEGMENTAÇÃO DE EXSUDATOS LIPÍDICOS EM IMAGENS DE RETINOGRAFIA

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia Elétrica do Centro de Tecnologia da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia Elétrica.

Orientador: Prof. Dr. Fabrício Gonzalez Nogueira.
Coorientador: Prof. Dr. Victor Hugo Costa de Albuquerque.

FORTALEZA

2026

Dados Internacionais de Catalogação na Publicação
Universidade Federal do Ceará
Sistema de Bibliotecas
Gerada automaticamente pelo módulo Catalog, mediante os dados fornecidos pelo(a) autor(a)

N385c Almeida Neto, Pedro Florencio de.
Comparação entre vision transformers e U-Net com MobileNetV2 na segmentação de Exsudatos Lipídicos em imagens de retinografia / Pedro Florencio de Almeida Neto. – 2026.
78 f. : il. color.

Trabalho de Conclusão de Curso (graduação) – Universidade Federal do Ceará, Centro de Tecnologia, Curso de Engenharia Elétrica, Fortaleza, 2026.
Orientação: Prof. Dr. Fabrício Gonzalez Nogueira.
Coorientação: Prof. Dr. Victor Hugo Costa de Albuquerque.

1. Visão computacional. 2. Segmentação semântica. 3. MobileViT. 4. Vision transformers. 5. Retinopatia diabética. I. Título.

CDD 621.3

PEDRO FLORENCIO DE ALMEIDA NETO

COMPARAÇÃO ENTRE VISION TRANSFORMERS E U-NET COM MOBILENETV2 NA SEGMENTAÇÃO DE EXSUDATOS LIPÍDICOS EM IMAGENS DE RETINOGRAFIA

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia Elétrica do Centro de Tecnologia da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia Elétrica.

Aprovada em: 26 de janeiro de 2026

BANCA EXAMINADORA

Prof. Dr. Fabrício Gonzalez Nogueira (Orientador)
Universidade Federal do Ceará (UFC)

Prof. Dr. Victor Hugo Costa de Albuquerque (Coorientador)
Universidade Federal do Ceará (UFC)

Eng. Alan Batista de Oliveira
Universidade Federal do Ceará (UFC)

Me. Renê Ripardo Calixto
Universidade Federal do Ceará (UFC)

Me. Alan Sergio dos Santos
Instituto Federal de Educação, Ciência e Tecnologia do Ceará (IFCE)

Ao meu pai, pelas lições que nenhum ambiente acadêmico poderia ensinar. A referência do seu esforço e a confiança em Deus foram fundamentais para a minha jornada.

AGRADECIMENTOS

À Deus por sempre me guiar e conduzir aos meus objetivos conforme Seu tempo.

Aos meus pais por todo esforço empenhado no começo da minha graduação e por fazerem o possível e impossível para me criarem.

Ao Prof. Dr. Victor Hugo, pelas oportunidades concedidas, incentivo e paciência.

Ao Prof. Dr. Fabrício Nogueira pela parceria e confiança.

Aos colegas profissionais participantes da banca examinadora Alan Sérgio, Alan Batista e Renê Ripardo pelo tempo despendido e pelas valiosas contribuições.

Aos colegas de turma Ramon, Rodrigo, Pedro e Raíssa, por toda ajuda e companheirismo durante a graduação.

Aos companheiros de projetos Caio, Clailton e Douglas, por todos os momentos vividos juntos e por todo trabalho em equipe.

Aos professores René, Raimundo, Raphael, Kléber e Arthur, do Departamento de Engenharia Elétrica, pela humanidade na docência e por valorizarem o indivíduo por trás do registro acadêmico.

Aos colegas de trabalho Juliana, Victor e Fernando por todo apoio e incentivo para a conclusão desse projeto.

À Hailla pelo amor traduzido em todo companheirismo e apoio, que foi fundamental para a finalização desse trabalho e da graduação.

À família Krulicoski pelo acolhimento e confiança durante meu último ano de graduação.

Ao Pablo, pela amizade e incentivo para continuar produzindo esse projeto.

Ao meu sobrinho, Arthur, pelas brincadeiras e momentos de descanso em meio às turbulências do último ano.

Àqueles que me abriram portas em bolsas e pesquisas, permitindo meu aprimoramento e amadurecimento técnico.

"Se você não gosta do seu destino, não o aceite.
Em vez disso, tenha a coragem de mudá-lo do
jeito que você quer que ele seja."
(KISHIMOTO, Masashi)

RESUMO

A Retinopatia Diabética é uma doença que acomete pessoas com Diabetes, que está entre os principais motivos de perda de visão entre indivíduos de 20 a 75 anos no Brasil. Com o aumento do número de casos de Diabetes no país e no mundo, novas tecnologias têm emergido com o objetivo de diagnosticar, via exame de fundoscopia, os primeiros indícios da doença. Dentre os fatores que caracterizam a doença, a presença de partículas brancas de lipídios, intitulados exsudatos lipídicos, são comuns e visualmente evidentes em imagens da retina. Este trabalho investigou o desempenho de duas abordagens no campo da Visão Computacional para a segmentação semântica de exsudatos duros, visando a promoção de uma análise comparativa de uma abordagem clássica e outra estado-da-arte otimizada para Unidades de Processamento Gráfico (GPUs) móveis, tendo em vista o avanço da pesquisa e desenvolvimento de aparelhos portáteis baseados em *smartphones* para exames de fundo de olho. Foram aplicados os modelos U-Net com uso de MobileNetV2 como *encoder* de sua arquitetura e MobileViT, uma proposição de *Vision Transformer* (ViT), focada em aplicação em dispositivos móveis. A partir dos dois modelos, foi realizada a comparação de 3 métricas amplamente utilizadas em tarefas de segmentação semântica na Visão Computacional – o Coeficiente de *Dice*, a Interseção sobre União (IoU) e o tempo de inferência, considerando a GPU NVIDIA Tesla T4, disponibilizada pelo Google Colab. O MobileViT apresentou melhores resultados que a U-Net, com coeficiente de *Dice* superando em 49% o modelo baseado em Rede Neural Convolucional (CNN), além de apresentar uma superioridade significante no âmbito da eficiência computacional. Ainda que superior nas métricas, o modelo de ViT apresentou um *overfitting*, mesmo com aplicação de técnica de *augmentation* das imagens. Os resultados evidenciam a superioridade computacional do MobileViT, apontando este modelo pré-treinado como uma solução promissora para a segmentação semântica eficiente de imagens médicas.

Palavras-chave: Retinopatia Diabética. Visão Computacional. Segmentação. CNN. *Vision Transformers*.

ABSTRACT

Diabetic Retinopathy is a disease affecting people with Diabetes, which is among the leading causes of vision loss in individuals aged 20 to 75 in Brazil. With the rising number of Diabetes cases both nationally and globally, new technologies have emerged aiming to diagnose the early signs of the disease via fundoscopy examinations. Among the factors characterizing the disease, the presence of white lipid particles, termed lipid exudates, is common and visually evident in retinal images. This study investigated the performance of two approaches in the field of Computer Vision for the semantic segmentation of hard exudates, aiming to promote a comparative analysis between a classical approach and a state-of-the-art one optimized for mobile GPU, given the advancement in research and development of portable smartphone-based devices for fundus examinations. The U-Net model using MobileNetV2 as its architectural encoder, and MobileViT, a proposed Vision Transformer (ViT) focused on applications in mobile devices, were applied. Based on these two models, a comparison was performed using 3 metrics widely employed in Computer Vision semantic segmentation tasks – the Dice Coefficient, the Intersection over Union (IoU), and inference time, considering the Nvidia Tesla T4 GPU provided by Google Colab. MobileViT presented better results than U-Net, with a Dice coefficient surpassing the CNN-based model by 49%, in addition to demonstrating significant superiority in terms of computational efficiency. Although superior in metrics, the ViT model exhibited overfitting, even with the application of image augmentation techniques. The results evidence the computational superiority of MobileViT, pointing to this pre-trained model as a promising solution for the efficient semantic segmentation of medical images.

Keywords: Diabetic Retinopathy; Computer Vision; Segmentation; CNN; Vision Transformers.

LISTA DE FIGURAS

Figura 1 – Comparação imagens de fundo de olho: (a) retina com sinais de retinopatia diabética e (b) exemplo sem lesões aparentes 19
Figura 2 – Paradigmas da Visão Computacional 21
Figura 3 – Tipos de segmentação de imagens 22
Figura 4 – Arquitetura LeNet-5 24
Figura 5 – Exemplo de filtro de convolução 25
Figura 6 – Funções de Ativação Unidade Linear Retificada (ReLU) e Função Exponencial Normalizada (softmax) 26
Figura 7 – *Max Pooling* com filtro de 2x2 e *stride* de 2 27
Figura 8 – Taxonomia de *Loss Functions* para Segmentação Semântica 28
Figura 9 – Arquitetura U-Net 30
Figura 10 – Arquitetura original de um ViT 32
Figura 11 – Arquitetura original do MobileViT 33
Figura 12 – Retinógrafo Kowa VX-10α 35
Figura 13 – Amostra de captura da retina (a) e máscara de exsudatos duros (b) 36
Figura 14 – Esquemático do fluxo das imagens 36
Figura 15 – Técnica de inversão horizontal 38
Figura 16 – Dice (a) e IoU (b) para 50 épocas de treinamento da U-Net 43
Figura 17 – Focal + Dice Loss para o modelo U-Net 43
Figura 18 – Amostra da segmentação semântica da U-Net 44
Figura 19 – Tversky Loss no treino e teste da MobileViT 45
Figura 20 – Dice (a) e IoU (b) para 27 épocas de treinamento da MobileViT 46

LISTA DE TABELAS

Tabela 1 – Classificação de Retinopatia Diabética 18
Tabela 2 – Dados da GPU NVIDIA Tesla T4 34
Tabela 3 – Configuração dos Hiperparâmetros de Treinamento do MobileViT 41
Tabela 4 – Comparativo de desempenho entre o MobileViT e a U-Net com MobileNetV2 46

LISTA DE ABREVIATURAS E SIGLAS

CNN	Rede Neural Convolucional
Colab	Google Colaboratory
GPU	Unidade de Processamento Gráfico
IDRiD	Indian Diabetic Retinopathy Image Dataset
IoU	Interseção sobre União
OMS	Organização Mundial da Saúde
ReLU	Unidade Linear Retificada
softmax	Função Exponencial Normalizada
ViT	*Vision Transformer*

LISTA DE SÍMBOLOS

e	Constante de Euler
$^{\circ}$	Ângulo em grau
α	Letra grega alfa
γ	Letra grega gama
β	Letra grega beta

SUMÁRIO

1 INTRODUÇÃO 15
1.1 Justificativa 15
1.2 Objetivos 16
1.2.1 Objetivo Geral 16
1.2.2 Objetivos Específicos 16
1.3 Estrutura do Trabalho 16
2 FUNDAMENTAÇÃO TEÓRICA 18
2.1 Retinopatia Diabética 18
2.2 Fundoscopia Dilatada 19
2.3 Retinógrafos Portáteis 19
2.4 Paradigmas da Visão Computacional 20
2.5 Segmentação de Imagens 21
2.5.1 Segmentação Semântica 21
2.5.2 Segmentação de Instâncias 22
2.5.3 Segmentação Panóptica 22
2.5.4 Avaliação de Resultados de Segmentação Semântica 22
2.6 Redes Neurais Convolucionais 23
2.6.1 Camada de Convolução 23
2.6.2 Função de Ativação 24
2.6.2.1 Unidade Linear Retificada *25*
2.6.2.2 Softmax *25*
2.6.3 Camada de Pooling 26
2.6.4 Camada Fully Connected 27
2.6.5 Loss Function 27
2.6.5.1 Binary Cross-Entropy Loss *27*
2.6.5.2 Focal Loss *28*
2.6.5.3 Dice Loss *29*
2.6.5.4 Tversky Loss *29*
2.6.6 Arquitetura U-Net 30
2.7 Vision Transformers 31

2.7.1 *MobileViT* 32
3 **METODOLOGIA** 34
3.1 **Plataforma Google Colaboratory** 34
3.2 **Conjunto de Dados de Imagens de Fundo de Olho** 34
3.3 **Upload das Imagens de Fundo de Olho** 35
3.4 **Pré-Processamento das Imagens** 36
3.5 **Preparação das Imagens para Modelagem** 37
3.5.1 *Transformação para Objeto tf.Data.Dataset* 37
3.5.2 *Tamanho de Lote (Batch)* 37
3.5.3 *Método Prefetch* 37
3.5.4 *Ampliação de Dados (Data Augmentation)* 37
3.6 **Modelagem da Arquitetura U-Net** 38
3.6.1 *Encoding por Transfer Learning* 38
3.6.2 *Fine Tuning* 39
3.6.3 *Decoding* 39
3.6.4 *Treinamento da U-Net com MobileNetV2 como Encoder* 39
3.7 **Modelagem por MobileViT** 40
3.7.1 *Ampliação de Dados* 40
3.7.2 *Repositório Hugging Face* 40
3.7.3 *Hiperparâmetros de Treinamento* 40
4 **RESULTADOS** 42
4.1 **Treinamento da U-Net com MobileNetV2** 42
4.2 **Treinamento da MobileViT** 45
4.3 **Comparação dos Modelos** 45
5 **CONCLUSÕES** 47
5.1 **Limitações** 47
5.2 **Trabalhos Futuros** 48
REFERÊNCIAS 50
APÊNDICE A –ARQUITETURA U-NET COM MOBILENETV2 . . . 52
APÊNDICE B – CÓDIGO-FONTE DA ARQUITETURA U-NET MODELADA 53

APÊNDICE C – CÓDIGO-FONTE DA ARQUITETURA MOBILEVIT MODELADA 66

1 INTRODUÇÃO

De acordo com o Ministério da Saúde e a Organização Mundial da Saúde (OMS), o número de pessoas no mundo que têm a visão prejudicada é cerca de 285 milhões, sendo entre 60% a 80% dos casos evitáveis. Somente no Brasil, 35 milhões de pessoas sofrem com algum tipo de dificuldade visual (IBGE, 2010). O diagnóstico oftalmológico depende muitas vezes da interpretação de imagens médicas, como retinografias ou tomografia de coerência óptica (OCT). De forma paralela, uma das doenças com tendência de crescimento no Brasil é a diabetes. Em estimativa apresentada pela Sociedade Brasileira de Diabetes, cerca de 20 milhões de pessoas sofrem com diabetes no Brasil, das quais 90% dos diabéticos do tipo 1 e 60% dos do tipo 2 terão algum grau de retinopatia diabética, doença ocular oriunda de uma complicação microvascular na retina, que afeta de 24 a 39% de pessoas com diabetes mellitus no Brasil e está entre as principais causas de perda de visão na faixa etária entre 20 e 75 anos. Estudos sugerem que o risco de cegueira pode ser reduzido para menos de 5% se a doença for diagnosticada e tratada precocemente. Entre as principais características da retinopatia diabética estão a presença de microaneurismas, alterações venosas, exsudatos lipídicos e hemorragia, de forma combinada ou não. A presença ou ausência de cada uma das características citadas trazem os níveis de gravidade da doença (Tabela 1). O presente estudo, compara abordagem clássica no campo da Visão Computacional e atual estado-da-arte para segmentação dos exsudados lipídicos, para fins de detecção automática de retinopatia diabética e apoio à classificação de nível de gravidade em imagens de retina. Dentre os modelos abordados, faz-se a aplicação de *Mobile Vision Transformers* para segmentação otimizada para GPUs móveis. As métricas de avaliação incluem Coeficiente de *Dice*, IoU e tempo de inferência.

1.1 Justificativa

A alta prevalência de doenças oculares e da diabetes mellitus no Brasil, além do tecnológico de equipamentos médicos portáteis, em especial os retinógrafos portáteis produzidos na academia ou já comercializados demandam uma oportunidade implementação de algoritmos de doenças oculares, que possibilita acelerar o diagnóstico por especialista, além de democratizar o acesso a exames de fundo de olho, possibilitando avanços no campo da telemedicina. Em pesquisa realizada na etapa de revisão de literatura, aparelhos como o Phelcom Eyer 2 e o retinógrafo portátil de baixo custo da "Mirabile Visu Startup Ltda"da Universidade de Passo

Fundo que utilizam a câmera de smartphone, demonstram oportunidade de implementação de abordagens de detecção de doenças oculares, demandando especialmente algoritmos leves e otimizados para inferência rápida e de baixo custo computacional. Tendo em vista esta necessidade específica, este trabalho visa comparar, o uso de *Vision Transformers*, arquitetura *Transformer* aplicada à imagens, que tem um potencial substancial para reconhecimento de imagens, segmentação e detecção de objetos em imagens ao modelo U-Net, composta por redes neurais convolucionais, tendo com objetivo obter a abordagem mais otimizada para a tarefa de segmentação de exsudatos duros, permitindo que os retinógrafos portáteis e existentes e os que forem desenvolvidos, usufruam dos algoritmos e abordagens apresentadas.

1.2 Objetivos

1.2.1 Objetivo Geral

Analisar e comparar o desempenho de abordagens algorítmicas baseadas em CNN e ViT, do campo da Visão Computacional, na tarefa de segmentação semântica de exsudatos lipídicos em imagens de fundo de olho, classificando os pixels relacionados ao fundo e aos depósitos de lipídio.

1.2.2 Objetivos Específicos

1. Segmentar exsudatos lipídicos em imagens de fundo de olho com abordagem de U-Net com MobileNetV2 como *encoder*;

2. Segmentar exsudatos lipídicos em imagens de fundo de olho com uso de ViT, a partir da arquitetura MobileViT;

3. Comparar o Coeficiente de Dice, IoU e tempo de inferência nas duas abordagens modeladas.

1.3 Estrutura do Trabalho

Este trabalho é composto por cinco capítulos. O Capítulo 1 apresenta o contexto da problemática, justificando o estudo e apresentando o objetivo geral e objetivos específicos alcançados. O Capítulo 2 apresenta os principais conhecimentos envolvidos no trabalho, desde a contextualização do campo da saúde à tecnologia de *hardware* e *software* da problemática

abordada. O Capítulo 3 detalha todo o processo metodológico da modelagem e avaliação das arquiteturas propostas para a segmentação semântica. O Capítulo 4 expõe os resultados das modelagens, por meio de gráficos e uma tabela comparativa. Por fim, o Capítulo 5 apresenta as conclusões e discussões sobre trabalhos futuros e as limitações contidas neste estudo.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Retinopatia Diabética

A retinopatia diabética (RD) é uma doença ocular oriunda de uma complicação microvascular na retina, que afeta de 24 a 39% de pessoas com diabetes *mellitus* no Brasil e está entre as principais causas de perda de visão na faixa etária entre 20 e 75 anos, segundo o Ministério da Saúde (BRASIL, 2021). Após 20 anos de doença, estima-se que 90% dos diabéticos de tipo 1 e 60% dos do tipo 2 terão algum grau de RD. Estudos internacionais indicam que o risco de cegueira pode ser reduzido para menos de 5%, se a RD for diagnosticada e tratada precocemente. Entre as principais características da doença estão a presença de microaneurismas, alterações venosas, exsudatos lipídicos (ou duros) e hemorragia, de forma combinada ou não. A presença ou ausência de cada uma das características citadas trazem os níveis de gravidade da doença, sendo as duas divisões principais a Retinopatia Diabética Não Proliferativa e a Retinopatia Diabética Proliferativa (Tabela 1). Os exsudatos duros são uma das características presentes na retinopatia diabética, e são descritos como depósitos de pequeno tamanho, brancos ou branco-amarelados com margens nítidas, que frequentemente possuem uma aparência cerosa ou brilhante/cintilante (GROUP, 1991).

Tabela 1 – Classificação de Retinopatia Diabética

Grau de Retinopatia Diabética	**Achados oftalmológicos observáveis**
RD Ausente	• Sem anomalias
RD Não Proliferativa Leve	• Microaneurismas
RD Não Proliferativa Moderada	• Microaneurismas • Hemorragias
RD Não Proliferativa Grave	• Hemorragias intrarretinianas • Anomalias microvasculares intrarretinianas • Exsudatos
RD Proliferativa	RD não proliferativa grave e 1 ou mais dos seguintes: • Neovascularização • Hemorragia vítrea/pré-retiniana

Fonte: Adaptado de CISNEROS-GUZMÁN *et al.* (2019)

2.2 Fundoscopia Dilatada

A fundoscopia, também conhecida como oftalmoscopia ou “fundo de olho”, é um exame feito para observar estruturas da parte interna dos olhos. São elas: o nervo óptico, a retina, e as veias e artérias responsáveis pelo fluxo sanguíneo da área (Figura 1). O exame serve para avaliar a situação clínica dos olhos e detectar alterações e doenças como glaucoma e câncer nos olhos. Além disso, auxilia no diagnóstico de doenças oculares e sistêmicas em recém-nascidos e serve também para acompanhar a evolução da diabetes mellitus e hipertensão arterial. A fundoscopia é um exame não invasivo, portanto, indolor. Normalmente, é necessária a aplicação de um colírio para a dilatação da pupila do paciente. O princípio básico da fundoscopia é a projeção de uma luz intensa nos olhos que, refletida pela retina, oferece a possibilidade de visualização das estruturas. Existem duas formas de realizar o exame. Na primeira, denominada fundoscopia direta, feita pelo clínico geral, obtém-se uma imagem aumentada quinze vezes, mas com uma cobertura mais restrita. Nesse caso, utiliza-se um aparelho portátil. A fundoscopia indireta é feita com equipamentos mais sofisticados e deve ser executada por um oftalmologista. Apesar da menor ampliação da imagem, é possível ver a retina de forma mais ampla.

Figura 1 – Comparação imagens de fundo de olho: (a) retina com sinais de retinopatia diabética e (b) exemplo sem lesões aparentes

(a) Retina com presença de exsudatos. (b) Retina normal.

Fonte: Adaptado de *Oclusões Retinianas, disponível em: <https://oftalmologistapedropaulo.com.br/2020/11/26/oclusoes-retinianas/>. Acesso em: 12 jan. 2026.*

2.3 Retinógrafos Portáteis

Visando democratizar o acesso a exames de fundo de olho e possibilitar a ampliação da telemedicina para o campo da oftalmologia, o desenvolvimento de dispositivos portáteis para

realização da fotografia da retina tem se intensificado nos últimos anos. Na revisão sistemática intitulada "Accuracy of low-cost, smartphone based retinal photography for diabetic retinopathy screening: A systematic review"(PRAYOGO *et al.*, 2023) os autores avaliam seis tipos de retinógrafos concebidos a partir dos *smartphones* iPhone 5, iPhone 5s e Samsung Galaxy S4, que visam a elaboração de um dispositivo de baixo custo para fotografia da retina. No Brasil, um projeto desenvolvido pela Fundação Universidade de Passo Fundo (UPF), por meio da *startup* Mirabile Visu Startup Ltda, realizaram o registro do desenho industrial de um modelo de retinógrafo portátil também baseado no uso de *smartphone* e um conjunto de lentes. Em notícia publicada pela assessoria de comunicação da Universidade de São Paulo (USP)[1], a empresa Phelcom, desenvolveu o retinógrafo portátil *Eyer*, que custa em torno de 5 mil dólares, valor abaixo de dispositivos vendidos em torno de 120 mil reais. Os dispositivos mencionados possuem como característica a aplicação de conjuntos de lentes em um aparelho móvel, que realiza a captura e tratamento da imagem da retina. O Phelcom Eyer, tem como particularidade sua natureza não-midriática, ou seja, dispensar o uso de colírio para dilatação da pupila, etapa comum em um exame de fundo de olho.

2.4 Paradigmas da Visão Computacional

A Visão Computacional é o campo da computação aplicada ao reconhecimento de imagens, identificação e segmentação de objetos, de maneira a assemelhar-se às tarefas realizadas pela visão humana. Essa ciência foi popularizada e formalmente instruída para alunos do MIT na década de 1970, onde os pesquisadores iniciaram suas produções em detecção e segmentação de bordas. David Marr, precursor dessa área de estudo e autor do livro "Vision"de 1982, introduziu o conceito de visão e destrinchou os métodos para compreensão de cenas por meio da Visão Computacional. O conceito de aprendizado profundo iniciou-se com o perceptron de Frank Rosenblatt, que buscou mimetizar um neurônio biológico (ROSENBLATT, 1958). Entretanto, foi com a introdução das CNNs na década de 90, que a tecnologia se tornou viável para a extração de características e interpretação complexa de imagens, como em uma retinografia (LECUN *et al.*, 1998). No contexto atual, o *pipeline* de Visão Computacional é comumente estruturado em quatro fases sequenciais: pré-processamento, simplificação, rotulagem e interpretação (MATIAS *et al.*, 2021). Conforme apresentadas na Figura 2, as fases podem ser subdividas em domínio do

[1] Para mais informações, acesse *Aparelho portátil permite diagnosticar doenças oculares a distância*, disponível em: <saocarlos.usp.br/aparelho-portatil-permite-diagnosticar-doencas-oculares-a-distancia/>. Acesso em: 05 jan. 2026.

Figura 2 – Paradigmas da Visão Computacional

Paradigm	Domain	Step/Action in the Processing Pipeline			
		I.Filtering and Pre-Processing	II.Conditioning and Simplifying	III.Labelling and Description	IV.Interpretation and Description
		Enhance the image	Simplify an image	Describe an image	Reason about an image
Classic Computer Vision	1.Value	Windowing and scale transform	Thresholding, arithmetic and logic operations with pixels	Histogram analysis	
	2.Space	Despeckling, mathematical morphology operations, convolutional and diffusion filters	Convolutional-based edge detection (Marr, Canny, Sobel, etc), saliency detection (Shaa'shua, etc), region-growing segmentation (watershed, Mumford-Shah, CSC, etc), skeletonization, split & merge	Spatial classifiers, classic ANNs (backpropagation, Boltzmann, Kohonen), statistic classifiers, classic machine learning (SVM, IBL, random forests, etc), rule-based classifiers, feature detection (SIFT, SURF, etc)	Spatial data structures (octrees, quadtrees), face-based models (DLFL, etc), boundary models, graph-based and spatial grammars, semantic networks
	3.Frequency	FFT- and wavelet-based filters (low-, band- and high-pass, etc)	FFT- and wavelet-based operations for frequency elimination, grouping and separation	FFT- and wavelet-based classification methods, Gabor filters	
Deep Learning	Convolutional Neural Networks	AlexNet, VGG, ResNet, Inception, YOLO, SSD, Mask-RCNN, etc			
Transform/Mapping		I → I	I → I	I → M	M → M

Fonte: MATIAS *et al.* (2021, p. 14)

valor, quando se envolvem matemáticas de pixel a pixel; domínio do espaço, quando o contexto dos pixels agrupados é analisado e, por fim, no domínio da frequência, em que são realizadas transformadas e filtros baseados no sinal de frequência das imagens.

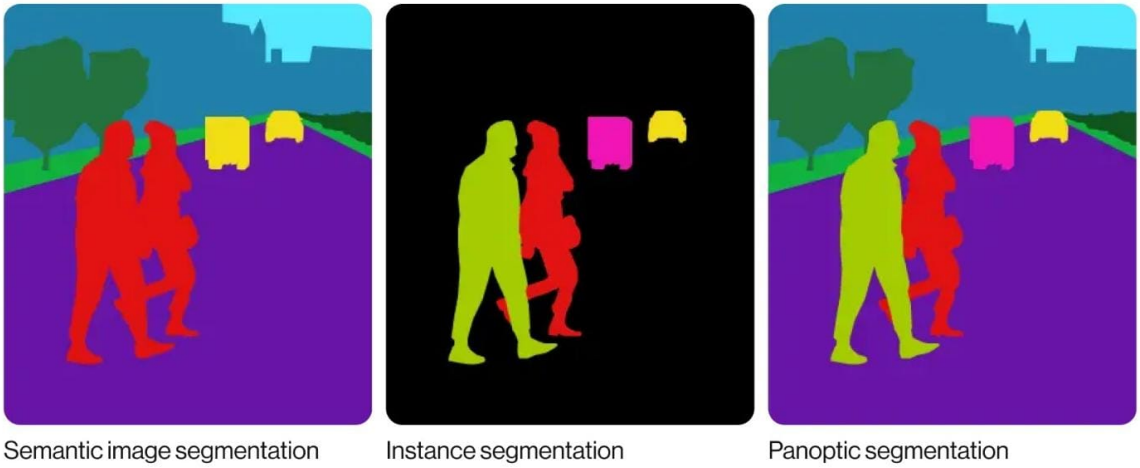
2.5 Segmentação de Imagens

Segmentação de imagens é o processo pelo qual, no âmbito de tarefas de visão computacional, há a separação de entes presentes em uma imagem. De forma geral, os entes podem ser objetos definidos ou elementos do ambiente da imagem. Há três tipos de segmentação de imagens – a segmentação semântica, segmentação de instâncias e segmentação panóptica (Figura 3). As subseções 2.5.1 a 2.5.3 detalham suas características principais.

2.5.1 Segmentação Semântica

A segmentação semântica é a tarefa de separação mais simples das três, sendo o objeto de estudo deste trabalho. Em modelos que realizam a segmentação semântica, o resultado é uma classificação da imagem, ao nível de píxel, sem realizar uma distinção do que é objeto e do que é ambiente.

Figura 3 – Tipos de segmentação de imagens



Fonte: *Instance Segmentation Explained: From Techniques to Practical Applications*, disponível em: <https://tech-stack.com/blog/instance-segmentation-computer-vision/>. Acesso em 13 jan. 2026

2.5.2 Segmentação de Instâncias

A segmentação de instâncias executa um passo mais difícil que a segmentação semântica, que é delinear o formato de cada instância de objeto de forma separada. Desta forma, mesmo quando há a sobreposição de objetos de mesma categoria, a segmentação de instâncias consegue realizar a separação e determinar os contornos de cada objeto, isolando-os entre si.

2.5.3 Segmentação Panóptica

Esse tipo de segmentação, por sua vez, combina as tarefas das outras citadas acima. A segmentação panóptica determina a que classe semântica o pixel pertence e diferencia cada instância de objeto contida em uma imagem, no seu processo de modelagem. A Figura 3 exemplifica os 3 tipos de segmentação de imagens.

2.5.4 Avaliação de Resultados de Segmentação Semântica

Os principais métodos analíticos para avaliação de uma segmentação de imagens são baseados em mensurar o quanto a máscara predita pelo modelo treinado se assemelha à máscara *ground-truth*. Com base nos pixels, pode-se realizar o cálculo por meio daqueles que são classificados correta ou erroneamente e realizar uma proporção dos acertos sobre o total classificado. Esta é a premissa para o cálculo da acurácia (Equação 2.1).

$$\text{Acurácia} = \frac{TP+TN}{TP+TN+FP+FN} \tag{2.1}$$

Onde:

- TP (*True Positive*): Verdadeiros Positivos;
- TN (*True Negative*): Verdadeiros Negativos;
- FP (*False Positive*): Falsos Positivos;
- FN (*False Negative*): Falsos Negativos.

Como as imagens de fundo de olho são caracterizadas por um alto desbalanceamento, onde os pixels de fundo são maioria, o valor da acurácia revela um desempenho que não reflete o comportamento do modelo quanto à classificação dos exsudatos. Nesse sentido, métricas como o Coeficiente de Dice (Equação 2.2) e IoU (Equação 2.3), conseguem penalizar o erro da máscara predita como maior rigor, uma vez que há um peso maior sobre a classe que se deseja prever.

$$\text{Dice} = \frac{2 \cdot TP}{2 \cdot TP+FP+FN} \tag{2.2}$$

$$\text{IoU} = \frac{TP}{TP+FP+FN} \tag{2.3}$$

2.6 Redes Neurais Convolucionais

As CNNs são uma evolução das redes neurais, desenvolvidas para o formato bidimensional de imagens. Baseado na arquitetura precedente LeNet-5 de LeCun *et al.* (1998), representada na Figura 4, pode-se descrever os componentes basilares de outras arquiteturas de CNNs.

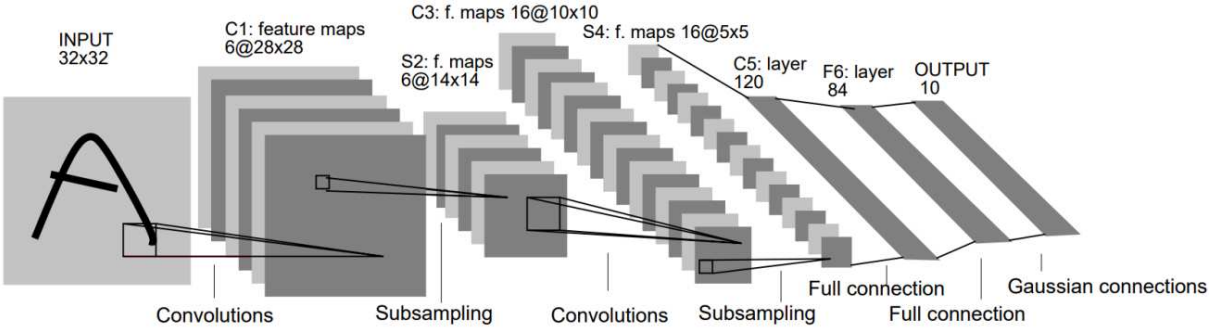
2.6.1 Camada de Convolução

As camadas de convolução presentes em uma arquitetura de CNNs são as responsáveis por identificar padrões nas imagens, baseada na operação de convolução (Equação 2.4). No contexto de CNN, a convolução é uma operação linear que atua como um filtro (*kernel*) que percorre o domínio da imagem.

Equação da convolução em uma imagem I:

$$(I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n) \tag{2.4}$$

Figura 4 – Arquitetura LeNet-5



Fonte: LECUN *et al. (1998, p. 2283)*

Onde:

- $I(i+m, j+n)$ representa o valor do pixel na região correspondente da imagem original;
- $K(m,n)$ representa o peso (valor) na matriz do filtro;
- $(*)$ representa o operador de convolução.

Conforme descrito, pode-se inferir que a convolução mede o grau de sobreposição ou similaridade entre duas funções, na qual, em uma imagem, é possível realçar padrões específicos ou aumentar ou diminuir sua nitidez. Por meio do exemplo da Figura 5 é possível identificar como a operação ocorre:

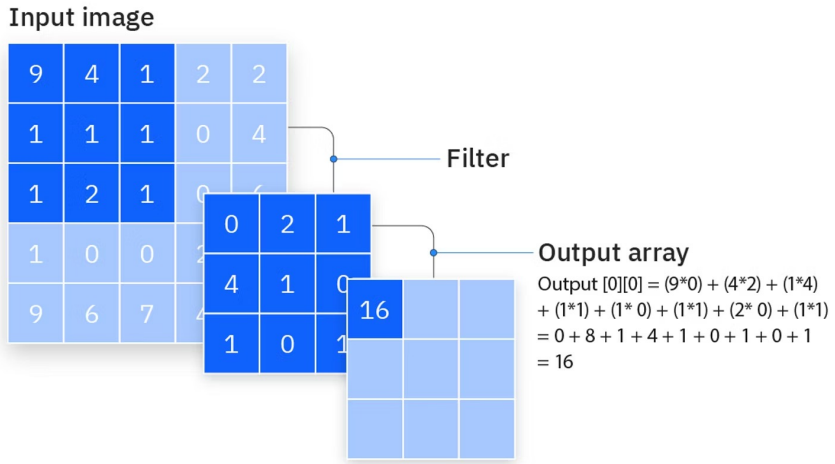
- Sobrepõe-se o filtro de convolução na imagem;
- Multiplica-se cada pixel da imagem pelos valores subsequentes do filtro;
- Soma-se os valores multiplicados da posição atual;
- Produzem-se os valores da matriz final;
- Desloca-se o filtro de convolução;
- Repete-se os passos anteriores.

Um conjunto de hiperparâmetros é adotado nas camadas de convolução, que geram resultados distintos entre si. O número de filtros, representa a quantidade de filtros distintos a serem implementados, determinando os canais da saída; o *stride*, é o número de pixels que o *kernel* desloca pela matriz de entrada; o *padding* é adição de pixels, geralmente de valor zero nas bordas da matriz de entrada, visando o controle do tamanho da saída.

2.6.2 Função de Ativação

A função de ativação é o elemento responsável por adicionar não linearidades ao modelo de CNN. Matematicamente, é uma transformação não linear que define a ativação de

Figura 5 – Exemplo de filtro de convolução



Fonte: *O que são redes neurais convolucionais?*, disponível em: <www.ibm.com/br-pt/think/topics/convolutional-neural-networks>. Acesso em: 06 jan. 2026.

neurônios em uma rede neural artificial, por meio de uma função diferenciável que permite a propagação posterior, aproveitando-se do gradiente e erro para atualização dos pesos e do viés. Uma rede sem função de ativação é essencialmente um modelo linear incapaz de resolver problemas complexos.

2.6.2.1 Unidade Linear Retificada

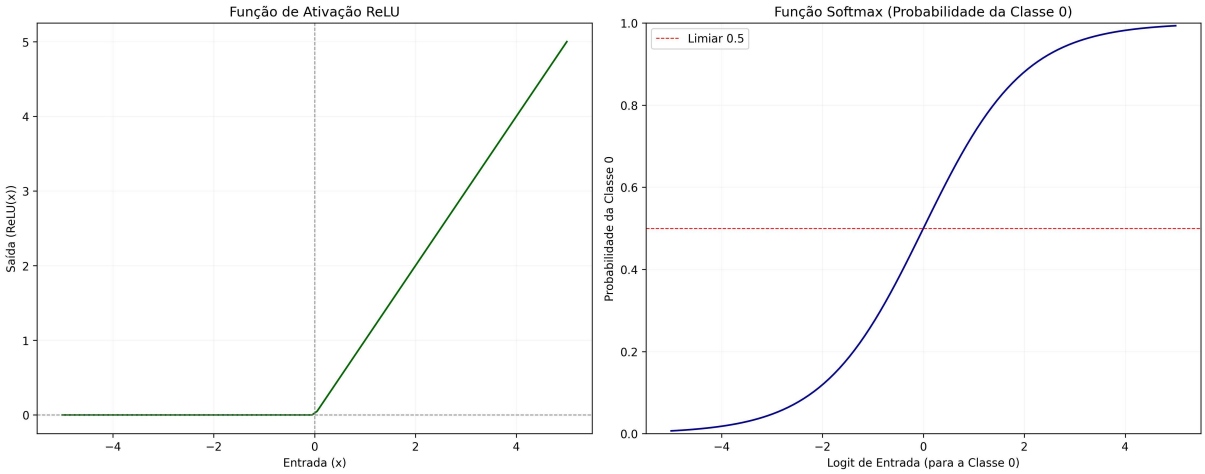
A ReLU é uma função de ativação comumente adicionada após os filtros de convolução. Ela tem como característica a repetição do valor da entrada quando este for positivo. Quando o valor é negativo, a saída da função é zero. A ReLU é descrita matematicamente na Equação 2.5. A característica matemática da ReLU possibilita que as redes neurais artificiais sejam modeladas com uma quantidade significativa de camadas sem um aumento expressivo da carga computacional, quando comparada com funções como a tangente hiperbólica ou a sigmóide.

$$f(x) = \max(0, x) \tag{2.5}$$

2.6.2.2 Softmax

A função de ativação softmax (Equação 2.6) realiza uma transformação em que a soma das saídas de uma camada é 1, ou seja, gera um resultado nos neurônios finais que varia de 0 a 1, podendo assumir uma probabilidade de pertencimento a classes, para o caso de uma tarefa

Figura 6 – Funções de Ativação ReLU e Softmax



Fonte: Elaborado pelo autor.

de reconhecimento de imagens.

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{2.6}$$

Onde:

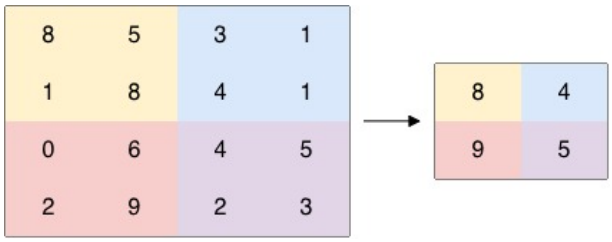
- $i = 1, \ldots, K$
- $\mathbf{z}$ é o vetor de entrada (logits).
- e é a constante de Euler (base do logaritmo natural).

Os componentes algébricos da Equação 2.6 definem os dois comportamentos principais do gráfico dessa função de ativação. A função exponencial aplicada a cada entrada assegura que os valores de saída da camada de neurônios sejam não-negativos, dando ênfase aos pesos de maior módulo. O denominador da softmax, representa uma normalização, na qual se somam os valores exponenciados e divide-se pelo total, garantindo que os valores de saída somados sejam exatamente 1, caracterizando as probabilidades de pertencimento às classes de imagem, por exemplo.

2.6.3 Camada de Pooling

A camada de *pooling* ou conjugação é a operação matemática que busca agrupar informações de um conjunto de pixels e transformar em uma única informação. Esse artifício permite reduzir a complexidade do modelo, aumentar a eficiência e limitar o risco de *overfitting*. De forma geral, eles se subdividem em "*max pooling*", *"min pooling*"e "*mean pooling*", quando

Figura 7 – *Max Pooling* com filtro de 2x2 e *stride* de 2



Fonte: Elaborado pelo autor.

extraem o valor máximo, mínimo e médio da região em questão, respectivamente. A Figura 7 exemplifica o *max pooling*.

2.6.4 Camada Fully Connected

A camada *fully connected* é a denominação de redes neurais artificiais que executam a tarefa de classificação, após todo o processo de reconhecimento de padrões em camadas anteriores. Geralmente, nessa etapa de modelagem é utilizada a função de ativação *softmax* para gerar um valor de probabilidade entre 0 e 1.

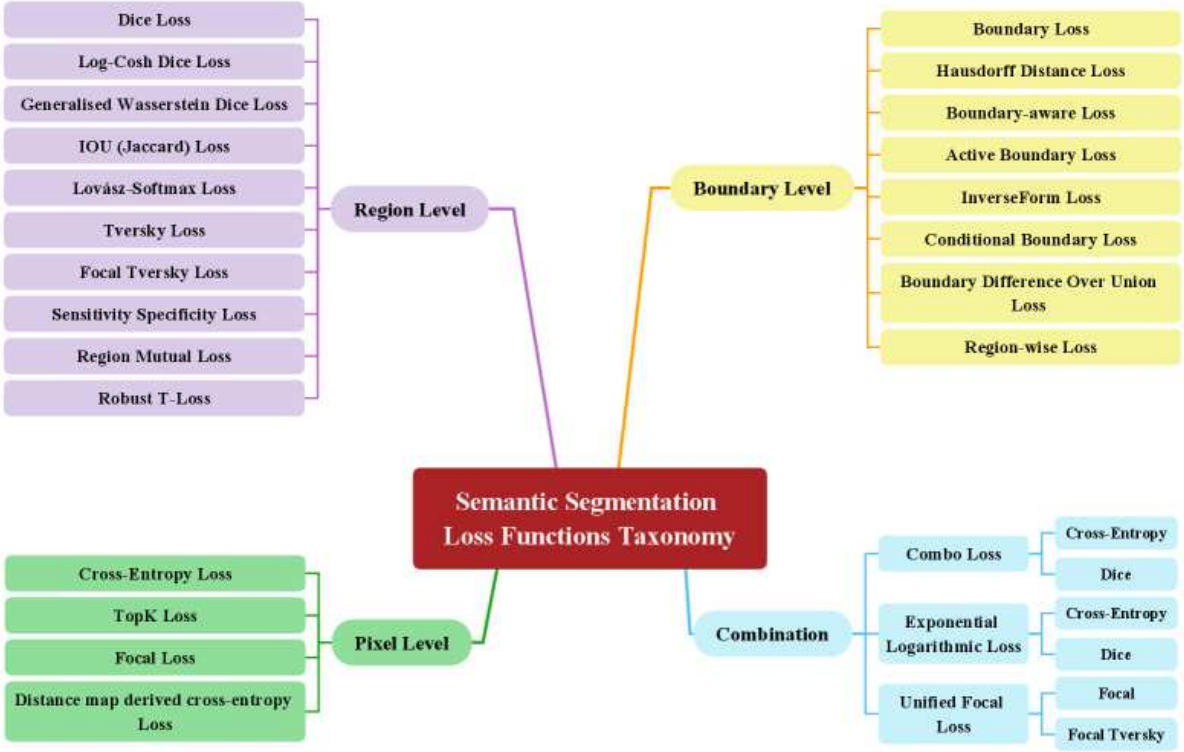
2.6.5 Loss Function

A *loss function*, ou função de perda, é o artifício matemático responsável por mensurar a distância entre o valor predito e o valor real em um modelo de aprendizado profundo. Devido a essa característica, as funções de perda orientam e direcionam o algoritmo para o ajuste dos pesos, de forma iterativa, de maneira a reduzir o valor da distância para o valor real. Na literatura, há uma taxonomia de funções de perda (AZAD *et al.*, 2023), que têm resultados favoráveis à modelos de segmentação semântica. Os subtópicos 2.6.5.1 a 2.6.5.4 explanam as funções de perda testadas durante o processo de treinamento dos modelos deste trabalho.

2.6.5.1 Binary Cross-Entropy Loss

Segundo AZAD *et al.* (2023), a Perda de Entropia Cruzada Binária, ou *Binary Cross-Entropy Loss*, é uma função de perda ao nível de pixel, ou seja, é calculado o erro de forma independente para cada pixel da imagem (Equação 2.7). Ela é baseada no conceito estatístico de entropia cruzada, na qual é medida a diferença entre duas distribuições de probabilidade. No contexto de segmentação semântica, em que há classificação binária, como, por exemplo, exsudato duro e fundo, a *Binary Cross-Entropy Loss* visa atribuir a cada pixel uma das duas

Figura 8 – Taxonomia de *Loss Functions* para Segmentação Semântica



Fonte: Azad *et al.* (2023, p. 2)

classes predefinidas e calcular o erro entre a previsão do modelo e os rótulos reais (*ground truth)*.

$$J = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{2.7}$$

Onde:

- N é o número total de amostras.
- y_i é o rótulo verdadeiro da amostra i (0 ou 1).
- $\hat{y}_i$ é a probabilidade prevista pelo modelo para a amostra i.
- log é o logaritmo natural.

2.6.5.2 *Focal Loss*

A *Focal Loss* surgiu para resolver o problema de desequilíbrio de classes em uma imagem. Em imagens médicas, como de uma retina, o total de pixels de elementos como exsudatos duros são substancialmente inferiores ao elemento de fundo. Neste sentido, a *Focal Loss* modifica a *Cross-Entropy Loss*, incorporando matematicamente o fator $(1-p_t)^{\gamma}$, que impede que os pixels de fundo, classificados com alto índice de confiança, dominem o gradiente

descendente durante o treinamento. A formulação matemática é dada pela Equação 2.8:

$$FL(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t) \tag{2.8}$$

Onde:

- $(1-p_t)^{\gamma}$: É o fator modulador que reduz a influência de exemplos fáceis;
- γ (gama): É o parâmetro de focalização (geralmente definido como 2);
- α_t (alfa): É um fator de ponderação utilizado para equilibrar a importância entre classes positivas e negativas.

2.6.5.3 Dice Loss

A Dice Loss é baseada no coeficiente de Dice (Equação 2.2). Ela otimiza diretamente a sobreposição espacial entre a máscara predita e a máscara real (*Ground Truth*). Esta característica a torna robusta ao desbalanceamento de classes, uma vez que o cálculo ignora a contagem de pixels de fundo, focando exclusivamente na qualidade da interseção da região de interesse. A função é definida como o complemento do coeficiente de Dice, conforme a Equação 2.9:

$$DL(y,\hat{y}) = 1 - \frac{2\sum_i y_i\hat{y}_i + \varepsilon}{\sum_i y_i + \sum_i \hat{y}_i + \varepsilon} \tag{2.9}$$

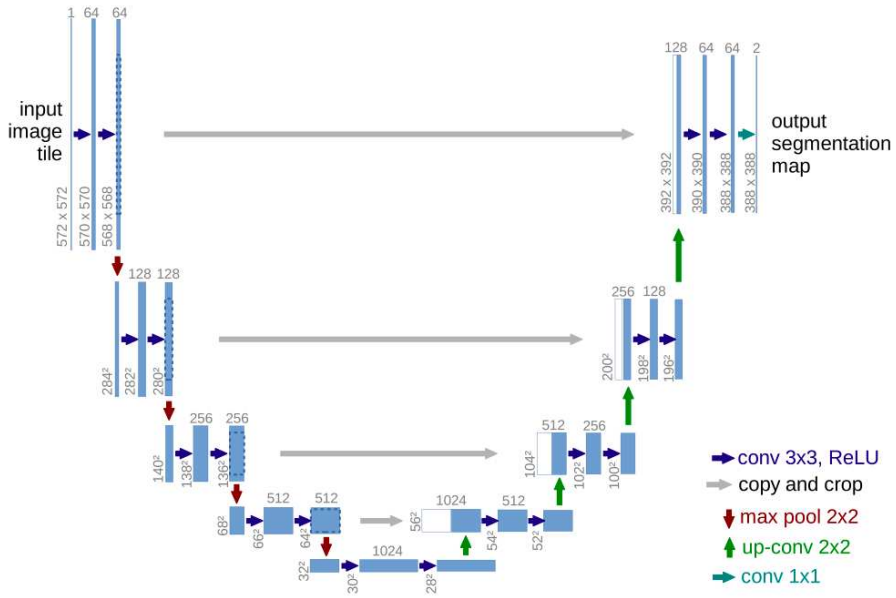
Onde:

- y_i: Valor do pixel na máscara real (0 ou 1);
- $\hat{y}_i$: Probabilidade predita para o pixel (saída da camada Sigmoid);
- ε (*smooth factor*): Fator de suavização (geralmente 1) para evitar divisão por zero e estabilizar o treinamento em casos de máscaras vazias.

2.6.5.4 Tversky Loss

A Tversky Loss é função de perda baseada na Dice Loss, projetada para oferecer um melhor equilíbrio entre as métricas precisão e sensibilidade. Enquanto a Dice Loss trata falsos positivos (FP) e falsos negativos (FN) com o mesmo peso, a Tversky Loss introduz hiperparâmetros α e β que permitem ajustar a penalidade para cada tipo de erro. No contexto deste estudo, onde deixar de detectar uma lesão (FN) é frequentemente mais grave do que um

Figura 9 – Arquitetura U-Net



Fonte: RONNEBERGER *et al.* (2015)

falso alarme (FP), a Tversky Loss permite forçar o modelo a priorizar a detecção de exsudatos ajustando-se $\beta > \alpha$. A função é descrita pela Equação 2.10:

$$TL(y,\hat{y}) = 1 - \frac{\sum_i y_i\hat{y}_i + \varepsilon}{\sum_i y_i\hat{y}_i + \alpha\sum_i(1-y_i)\hat{y}_i + \beta\sum_i y_i(1-\hat{y}_i) + \varepsilon} \tag{2.10}$$

Onde:

- $\sum(1-y_i)\hat{y}_i$: Representa os Falsos Positivos (FP);
- $\sum y_i(1-\hat{y}_i)$: Representa os Falsos Negativos (FN);
- α e β: Hiperparâmetros de controle que ajustam o peso dos erros (geralmente $\alpha+\beta=1$). Caso $\alpha=\beta=0.5$, a Tversky Loss torna-se equivalente à Dice Loss.

2.6.6 Arquitetura U-Net

O algoritmo U-Net foi originalmente construído para aplicações de segmentação de imagens biomédicas, especialmente para o campo da citologia (RONNEBERGER *et al.*, 2015). O U-Net foi pensado para ser treinado em casos em que a quantidade de imagens seja baixa. Por se assemelhar à letra "U"(Figura 9), a U-Net carrega essa nomenclatura, que também descreve a metodologia de interação entre suas camadas.

A arquitetura U-NET pode ser subdividida em 2 componentes principais: o *encoding* e o *decoding*. A etapa de *encoding* é responsável por reduzir a quantidade de informação em

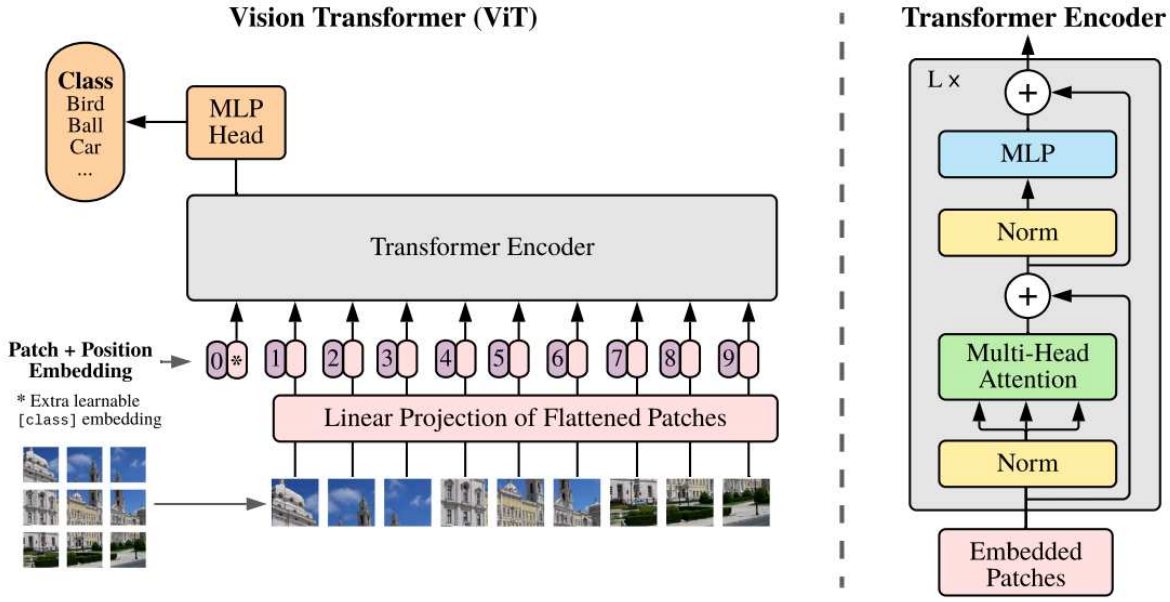
conjunto com o aumento da dimensionalidade da imagem, ou seja, a expansão do número de canais. Ela age como uma CNN tradicional, aplicando sequências de convoluções e *pooling*. Podem ser utilizados arquiteturas de CNN nesta etapa, como a MobileNetV2, aplicada no presente estudo. A etapa de *decoding*, por sua vez, é responsável por aumentar a quantidade de informação das imagens e redução dos canais, através de operações de *upsampling* (superamostragem) e convoluções. A interação entre as duas camadas se dá através das *skip connections*, que as conecta e permite que as informações de alta resolução, que seriam perdidas durante o *pooling*, consigam ser aplicadas no elemento *decoder*, permitindo o reconhecimento de detalhes como bordas e texturas e a geração de máscaras de segmentação precisas, ainda que treinadas com um *dataset* pequeno.

2.7 Vision Transformers

O ViT representa uma mudança de paradigma na Visão Computacional ao adaptar a arquitetura baseada em *Attention* — predominante no Processamento de Linguagem Natural (NLP) — para tarefas de reconhecimento visual (DOSOVITSKIY *et al.*, 2020). Diferentemente das CNNs, que processam pixels vizinhos via filtros locais (convoluções), o ViT trata a imagem como uma sequência linear de dados, permitindo a modelagem de dependências globais desde as primeiras camadas. O funcionamento do ViT pode ser sintetizado em três etapas fundamentais:

- Fragmentação em Patches: A imagem de entrada $x \in \mathbb{R}^{H \times W \times C}$ é dividida em uma sequência de N blocos fixos não sobrepostos (*patches*) de dimensão $P \times P$. Cada patch é tratado analogamente a um *token* (palavra) em uma sentença;
- Projeção Linear e Embeddings: Os patches são achatados (*flattened*) em vetores e projetados em um espaço latente de dimensão D. Adicionalmente, são somados vetores de posição (*Positional Embeddings*) para preservar a informação espacial, visto que a arquitetura Transformer é agnóstica à estrutura 2D da imagem;
- Mecanismo de Autoatenção (*Self-Attention*): O núcleo do modelo é o *Multi-Head Self-Attention* (MSA). Este mecanismo permite que cada patch atenda a todos os outros patches da imagem simultaneamente, capturando o contexto global instantaneamente, ao contrário das CNNs que necessitam de múltiplas camadas profundas para expandir seu campo receptivo.

Figura 10 – Arquitetura original de um ViT



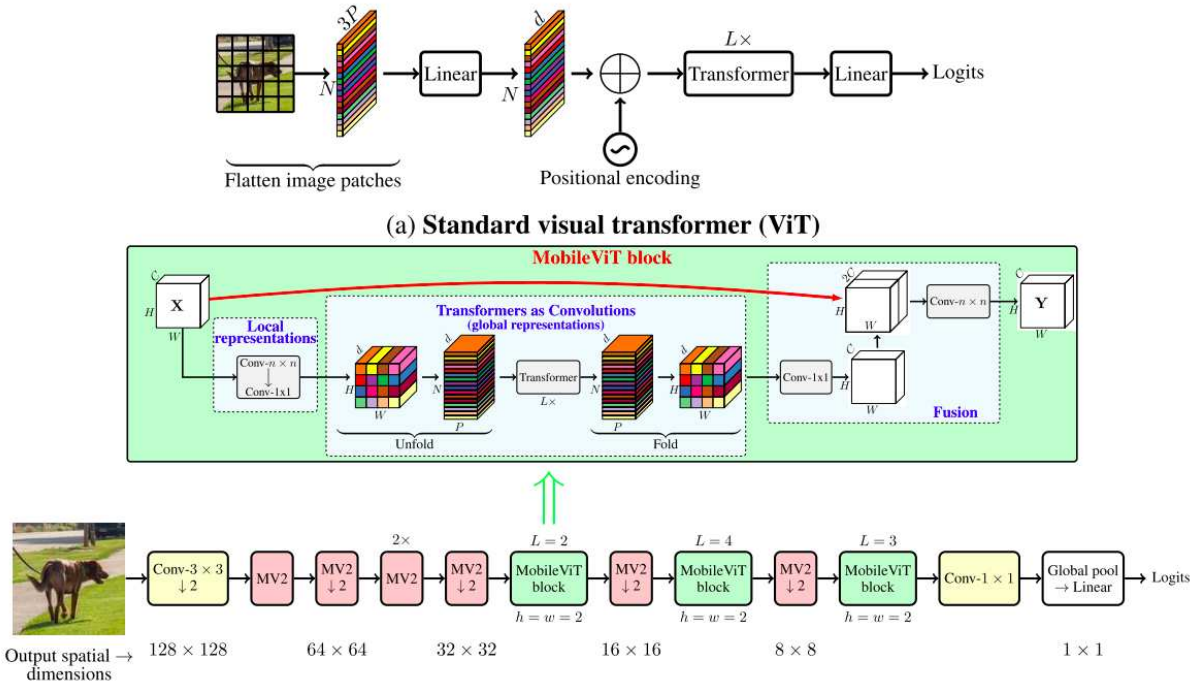
Fonte: Dosovitskiy *et al.* (2020)

2.7.1 MobileViT

O MobileViT (MEHTA; RASTEGARI, 2021), é uma arquitetura híbrida projetada para superar as limitações de latência e tamanho dos ViT tradicionais, tornando-os viáveis para dispositivos móveis. A premissa central do MobileViT é combinar as características das CNNs com as dos *Transformers*. Diferentemente de outras abordagens que inserem blocos de atenção no final da rede, o MobileViT introduz o conceito de aprendizado de representações globais esparsas através do Bloco MobileViT, que permite ao modelo analisar a imagem de forma completa com baixo custo computacional:

- Representação Local (Convoluções): Inicialmente, o bloco aplica uma convolução padrão $n \times n$ seguida por uma convolução ponto-a-ponto (1×1). O objetivo é capturar características espaciais de baixo nível (como bordas dos exsudatos) com a eficiência de uma CNN;
- Codificação Global (Transformer): Para aprender o contexto global, o mapa de características é transformado. Em vez de achatar a imagem inteira, como propõe o ViT, o MobileViT utiliza uma técnica de *Unfolding* para converter o tensor espacial em sequências de *tokens*, preservando a ordem dos patches. Estes tokens são processados por uma pilha de Transformers leves, permitindo que cada pixel tenha acesso à informação de pixels distantes;

Figura 11 – Arquitetura original do MobileViT



Fonte: MEHTA; RASTEGARI (2021)

– Fusão e Reconstrução (Folding): A saída do Transformer é redobrada (*Folding*) para voltar ao formato espacial original ($H \times W \times C$) e concatenada com as características de entrada originais via uma convolução de fusão.

Essa arquitetura permite que o MobileViT tenha um campo receptivo global efetivo desde as primeiras camadas da rede, diferentemente das CNNs puras, mantendo o número de parâmetros drasticamente menor que um ViT puro.

3 METODOLOGIA

A metodologia da segmentação de exsudatos lipídicos foi realizada de maneira a produzir uma análise comparativa de 2 paradigmas distintos de Visão Computacional. O primeiro, baseado em CNNs, buscou a construção de uma *baseline*, ou seja, um modelo base amplamente utilizado nos últimos anos para a segmentação semântica, e compará-lo com um modelo em estado-da-arte com a característica de ser leve, para potencial de inferência em GPUs móveis, uma vez que as atuais tecnologias portáteis de retinógrafos são baseados em *smartphones*.

3.1 Plataforma Google Colaboratory

Empregou-se a plataforma Google Colaboratory (Colab) para desenvolvimento dos algoritmos, treinamento e inferência dos modelos de segmentação semântica nas imagens de exsudatos duros. O Colab é um serviço em nuvem que possibilita, de forma remota, o uso de uma GPU que acelera e otimiza o processo de treinamento dos modelos de Inteligência Artificial. Os dados técnicos da GPU estão descritos na Tabela 2.

Tabela 2 – Dados da GPU NVIDIA Tesla T4

Característica	**Valor**
Nome da GPU	Tesla T4
Versão do Driver	550.54.15
Versão CUDA	12.4
Memória Total	15360MiB

Fonte: Google Colaboratory

3.2 Conjunto de Dados de Imagens de Fundo de Olho

O conjunto de dados (*dataset*) selecionado para a implementação e avaliação dos modelos foi o Indian Diabetic Retinopathy Image Dataset (IDRiD) (PORWAL *et al.*, 2018). O IDRiD é composto por 81 imagens de retina com resolução de 4288×2848 pixels, adquiridas por meio do retinógrafo *Kowa VX-10*α (Figura 12). As imagens foram organizadas em dois diretórios distintos: um contendo 54 imagens destinadas exclusivamente ao treinamento dos modelos e outro com 27 imagens reservadas para a etapa de teste, utilizada na avaliação de desempenho por meio das métricas definidas neste trabalho. Na Figura 13, está disposta uma

Figura 12 – Retinógrafo Kowa VX-10α

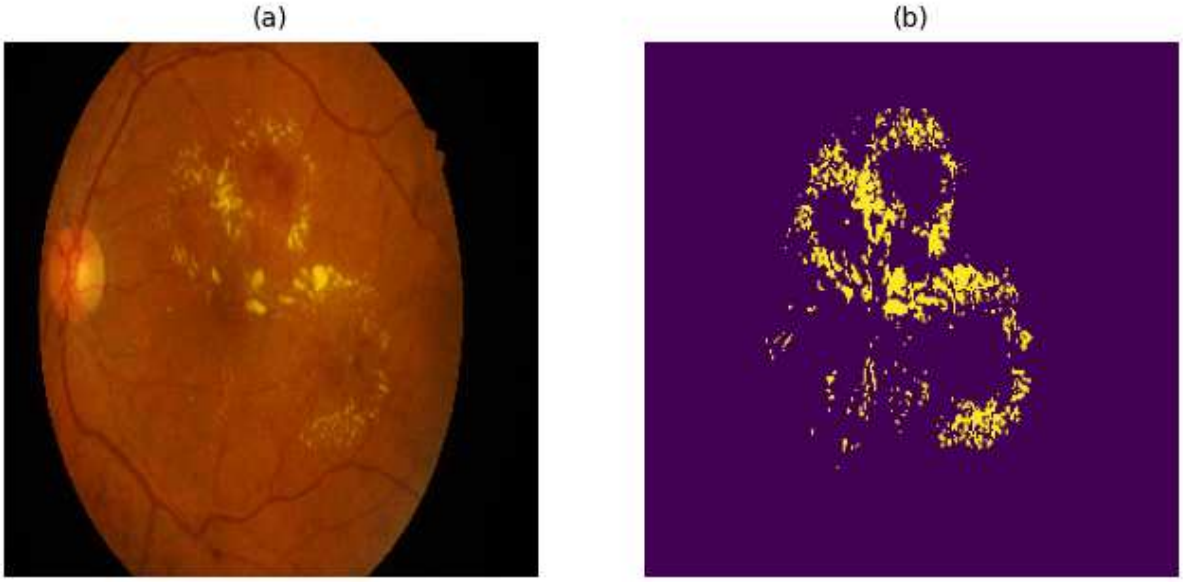
Fonte: Kowa American Corporation. Disponível em: <*https://ophthalmic.kowa-usa.com/discontinued/vx-10-alpha-mydriaticnon-mydriatic-combination-retinal-camera-features/*>. Acesso em: 11 jan. 2026.

amostra contendo a imagem de fundo de olho e sua respectiva máscara *ground-truth*. O IDRiD foi escolhido por conter anotações *ground-truth* das principais lesões associadas à Retinopatia Diabética presentes na retina, em especial, dos exsudatos duros, objeto de estudo deste trabalho. Além deles, microaneurismas, hemorragias e exsudados moles também estão documentados e segmentados por máscara binária. A estrutura do *dataset* é composta pela divisão em 3 sub-tarefas principais – segmentação, classificação e localização, facilitando a modelagem de algoritmos pelos pesquisadores. A população em que foi realizada os exames de fundo de olho são pacientes diabéticos de uma clínica no país.

3.3 Upload das Imagens de Fundo de Olho

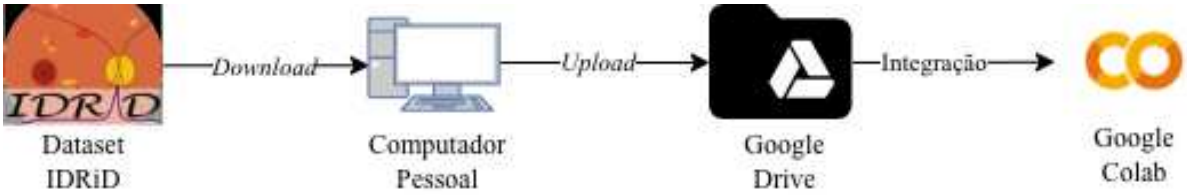
Para *upload* das imagens de fundo de olho, primeiramente foram carregadas as retinografias contidas no *dataset* IDRiD em uma pasta no Google Drive. A partir disso, foi introduzido, na plataforma Google Colab, o *snippet* de código responsável pela integração com a conta Google Drive em que as imagens foram armazenadas. Depois, realizou-se a leitura das imagens e máscaras por meio do método `tf.io.read_file`, para posteriormente realizar o

Figura 13 – Amostra de captura da retina (a) e máscara de exsudatos duros (b)



Fonte: Adaptado de PORWAL *et al.* (2018)

Figura 14 – Esquemático do fluxo das imagens



Fonte: Elaborado pelo autor.

pré-processamento (Seção 3.4).

3.4 Pré-Processamento das Imagens

Foi realizada a conversão dos arquivos de máscaras *ground-truth*, que inicialmente estão no formato *tiff* para o formato *png*, uma vez que, a *framework* Tensorflow, empregada em toda etapa de pré-processamento das imagens, não suporta esse formato nativamente pelas principais rotinas de decodificação da biblioteca. Tal característica pode introduzir maior complexidade no pipeline de pré-processamento e impactar o desempenho durante o treinamento dos modelos. Após isso, foi realizado o carregamento e adequação das imagens, seguindo a pipeline descrita no Algoritmo 1. O redimensionamento aplicado foi de 256x256 pixels e para a máscara foi realizada a binarização que classificou como 1 o valor relacionado aos pixels dos exsudatos duros e 0 ao valor do fundo da imagem, utilizando-se da interpolação por região mais

próxima, para garantir apenas valores inteiros (0 ou 1) nas máscaras.

Algoritmo 1: Pré-processamento de Imagem e Máscara

Entrada: caminhos da imagem p_{img} e da máscara p_{mask}, dimensão alvo N.

Saída: imagem normalizada e máscara binária. Leitura da imagem a partir de p_{img}, seguida de redimensionamento para $N \times N$ e normalização dos valores de pixel para o intervalo $[0,1]$. Leitura da máscara a partir de p_{mask}, redimensionamento para $N \times N$ e binarização, atribuindo valor 1 aos pixels da lesão e 0 ao fundo.

3.5 Preparação das Imagens para Modelagem

3.5.1 Transformação para Objeto tf.Data.Dataset

Foi implementado um bloco de código utilizando a API *tf.data*, que permitiu o agrupamento de imagens e a realização de um pipeline de treinamento feito por lotes, para fins de otimização de memória e recursos de computação. A partir dos tensores representados pela lista dos caminhos das imagens no Google Drive, emparelharam-se as imagens às suas respectivas máscaras e criou-se um objeto do tipo *tf.Data.Dataset*. Após isso, aplicaram-se, executando paralelismo de threads, as transformações explicadas no tópico 3.4.

3.5.2 Tamanho de Lote (Batch)

Optou-se pelo uso de 4 imagens por lote, pois dessa forma, é facultado o uso de GPUs com menor tamanho de memória e para que, possa-se usar imagens de maior resolução sem ultrapassar o limite de memória disponibilizada pelo ambiente de desenvolvimento.

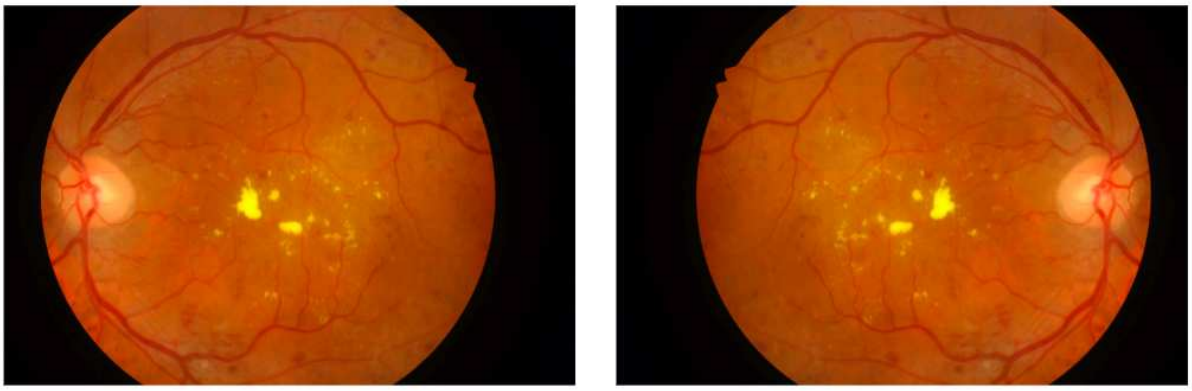
3.5.3 Método Prefetch

Visando otimizar o desempenho da *pipeline* de processamento adotada, aplicou-se o método Prefetch da *framework* Tensorflow, que possibilitou com que, enquanto a GPU está treinando com um lote de dados, a CPU possa processar o próximo lote em segundo plano.

3.5.4 Ampliação de Dados (Data Augmentation)

Foi aplicada ao conjunto de imagens de treinamento uma transformação geométrica do tipo inversão horizontal (*horizontal flip*), com probabilidade de aplicação fixada em 50%. Essa

Figura 15 – Técnica de inversão horizontal



Fonte: Elaborado pelo autor.

técnica de *data augmentation* foi empregada visando aumentar a variabilidade do conjunto de dados, reduzir o risco de sobreajuste (*overfitting*) e promover maior capacidade de generalização dos modelos treinados. A Figura 15 ilustra um exemplo de inversão horizontal aplicada a uma amostra do conjunto de imagens.

3.6 Modelagem da Arquitetura U-Net

Foi aplicada a arquitetura U-Net, elaborada para exigir um conjunto de treinamento mínimo e extrair o máximo de informações para a segmentação semântica em uma imagem. Por ser um modelo elaborado para menores conjuntos de imagens e para o campo da saúde, a escolha dessa CNN foi realizada visando ser uma *baseline* para outros modelos baseados em CNN em trabalhos posteriores.

3.6.1 Encoding por Transfer Learning

A etapa de *encoding* ou subamostragem é responsável por promover uma redução da quantidade de informação da imagem, ou seja, diminuição da sua resolução espacial, e, de forma concomitante, um aumento da dimensionalidade, promovendo a criação de novos canais. Foi inserido como *encoder* o modelo pré-treinado MobileNetV2 com imagens do banco de imagens ImageNet, excluindo as camadas finais, responsáveis pela classificação do modelo, utilizando-se das camadas intermediárias para extração de *features*.

3.6.2 Fine Tuning

O *fine tuning*, ou ajuste fino, foi realizado visando a obtenção de um modelo robusto e personalizado para a segmentação proposta. Portanto, aproveitou-se o treinamento do modelo MobileNetV2 por *transfer learning* explicado no tópico 3.6.1 e descongelou-se as 30 últimas camadas, mais próximas da saída do *encoder*, para serem ajustadas durante o novo processo de treinamento.

3.6.3 Decoding

O processo de *decoding* ou superamostragem, realizou o aumento da quantidade de dados e redução da dimensionalidade da imagem. Desta forma, as máscaras de segmentação foram reconstruídas a partir dos mapas de características comprimidos do *encoder*. Ele fez isso via camadas de up-sampling, ou aumento de resolução. Utilizou-se como número de filtros, os valores 512, 256, 128, 64, linearmente.

3.6.4 Treinamento da U-Net com MobileNetV2 como Encoder

O treinamento do modelo U-Net, especificado nos tópicos anteriores, foi parametrizado com os seguintes campos:

- Otimizador: Escolheu-se o otimizador Adam (Adaptative Moment Estimation) por sua popularidade e eficiência em trabalhos com redes neurais convolucionais. Com o uso de algoritmos RMSProp e Adagrad, ele adapta a taxa de aprendizado para cada peso individual da rede.
- *Learning Rate*: A taxa de aprendizado aplicada foi de 0,0001, o que proporcionou uma conversão de forma mais estável nos ajustes dos pesos do modelo a cada lote de imagens.
- Função de Perda: A função de perda aplicada foi projetada para lidar com o desbalanceamento entre classes, uma vez que há muito mais pixels de fundo do que pixels da classe de interesse. Aplicou-se a *focal loss*, com parâmetros α e γ iguais a 0,8 e 2, respectivamente. O uso desta função de perda visou ponderar pixels que são mais difíceis de classificar corretamente, diminuindo-se o peso de pixels mais comuns, onde o modelo tende a acertar com maior confiança. Combinou-se uma perda baseada no Coeficiente de Dice para o incentivo a uma ideal sobreposição geral da máscara.

3.7 Modelagem por MobileViT

3.7.1 Ampliação de Dados

Visando a melhoria do desempenho do algoritmo MobileViT, a etapa de *data augmentation* adotada foi ampliada, se comparada ao modelo anterior. Foram adotados, além da inversão horizontal, a inversão vertical, rotação com múltiplos de 90°, o deslocamento vertical e horizontal da imagem em até 6,25%, redimensionamento do tamanho em até 10%, além de alterações aleatórias no brilho e contraste, bem como distorções na grade da imagem. Os parâmetros adotados em cada uma das funções que realizam as transformações no *dataset* podem ser consultados no Apêndice C.

3.7.2 Repositório Hugging Face

Foi utilizado o modelo pré-treinado disponibilizado pelos pesquisadores desenvolvedores na plataforma Hugging Face, por meio de sua API oficial. O modelo foi treinado com o *dataset* ImageNet-1k, que consiste em 1 milhão de imagens contendo 1000 classes.

3.7.3 Hiperparâmetros de Treinamento

Foi realizada a instanciação do modelo pré-treinado *"apple/mobilevit-small"*, aplicando o parâmetro *"num_labels"* como 2, uma vez que se tratou de uma classifição binária – o pixel pertence à classe de exsudato duro ou fundo. Após a instanciação do modelo, definiu-se o peso para as classes, para lidar com o desbalanceamento, uma vez que as áreas de exsudatos são significativamente menores que as áreas de fundo do olho. O parâmetro *"class_weights"* escolhido foi de 10. Desta forma, o erro na predição de um exsudato foi penalizado em 10 vezes quando comparado a um erro de previsão de fundo. Os hiperparâmetros principais do modelo estão descritos na Tabela 3.

Tabela 3 – Configuração dos Hiperparâmetros de Treinamento do MobileViT

Hiperparâmetro	**Configuração**
Épocas	30
Função de Perda	Tversky ($\alpha = 0,3; \beta = 0,7$)
Otimizador	AdamW ($lr = 1e-4, weight_decay = 1e-5$)
Scheduler de Taxa de Aprendizado	Cosine Annealing (T_max=30)
Patience do *Early Stopping*	7 Épocas

Fonte: Elaborado pelo autor.

4 RESULTADOS

Neste capítulo, são apresentados os resultados da segmentação semântica realizada com U-Net e com MobileViT. Visando a comparação do desempenho dos dois modelos, foi realizado a produção de gráficos da função de perda ao longo do treinamento, por meio da biblioteca `Matplotlib` do Python, além do cálculo das métricas coeficiente de Dice, IoU e tempo de inferência.

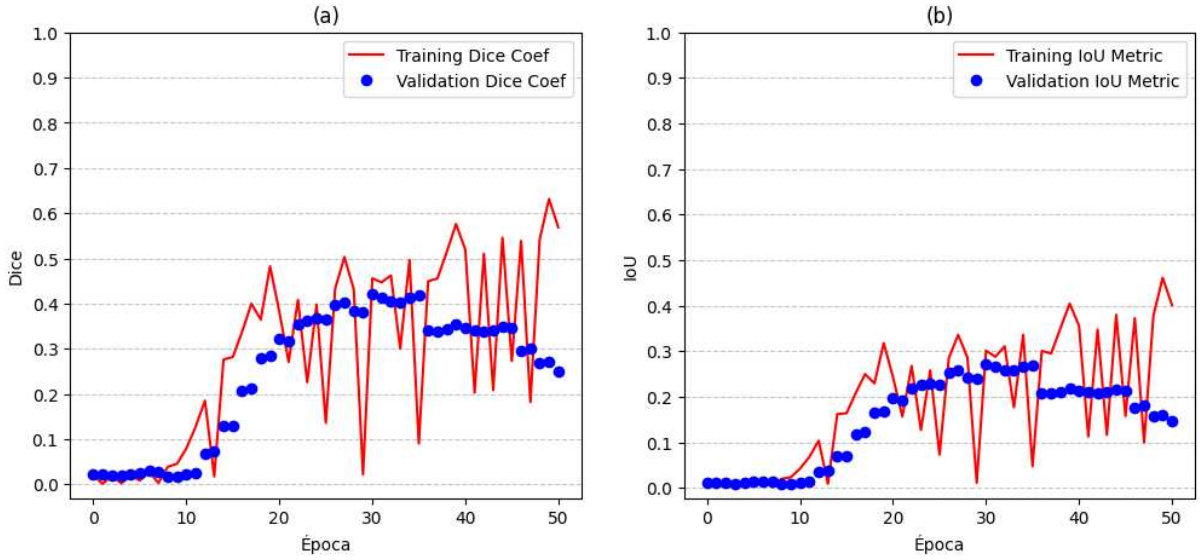
4.1 Treinamento da U-Net com MobileNetV2

O treinamento da U-Net com MobileNetV2 apresentou valores de Dice e IoU consideravelmente baixos ao longo das épocas (Figura 16). No contexto da segmentação semântica de imagens médicas, métricas como Dice e IoU assumem valores elevados apenas quando há alta correspondência pixel a pixel entre a predição do modelo e a máscara de referência. Valores inferiores a 0,5 são geralmente interpretados como baixos na literatura, indicando limitada sobreposição entre as regiões segmentadas.

A função de perda adotada, composta pela combinação de *focal loss* e *dice loss*, evidenciou a ocorrência de sobreajuste do modelo, uma vez que o desempenho obtido no conjunto de teste foi significativamente inferior ao observado durante o treinamento. Uma amostra de quatro resultados da segmentação semântica em imagens de retinopatia diabética pode ser observada na Figura 18. Conforme observado, o modelo foi capaz de identificar corretamente regiões correspondentes a exsudatos lipídicos com maior evidência visual; contudo, pixels associados a lesões de menor contraste ou reduzida extensão espacial foram predominantemente classificados como fundo. Esse comportamento pode estar diretamente relacionado à redução da resolução das imagens durante a etapa de pré-processamento do modelo, a qual limita a preservação de detalhes espaciais relevantes para a correta segmentação dessas estruturas.

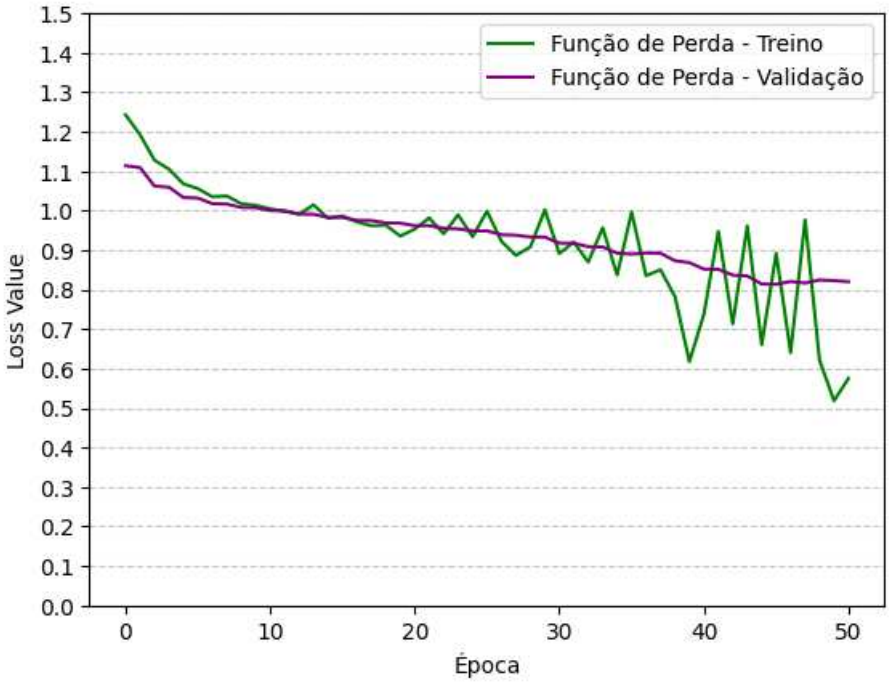
Adicionalmente, observa-se que as curvas das métricas Dice e IoU ao longo do treinamento apresentaram comportamento instável, com variações significativas entre épocas consecutivas. Em contrapartida, a função de perda exibiu uma redução contínua e acentuada no conjunto de treinamento, enquanto no conjunto de teste essa queda ocorreu de forma mais suave, indicando possível dificuldade de generalização do modelo para dados não vistos.

Figura 16 – Dice (a) e IoU (b) para 50 épocas de treinamento da U-Net



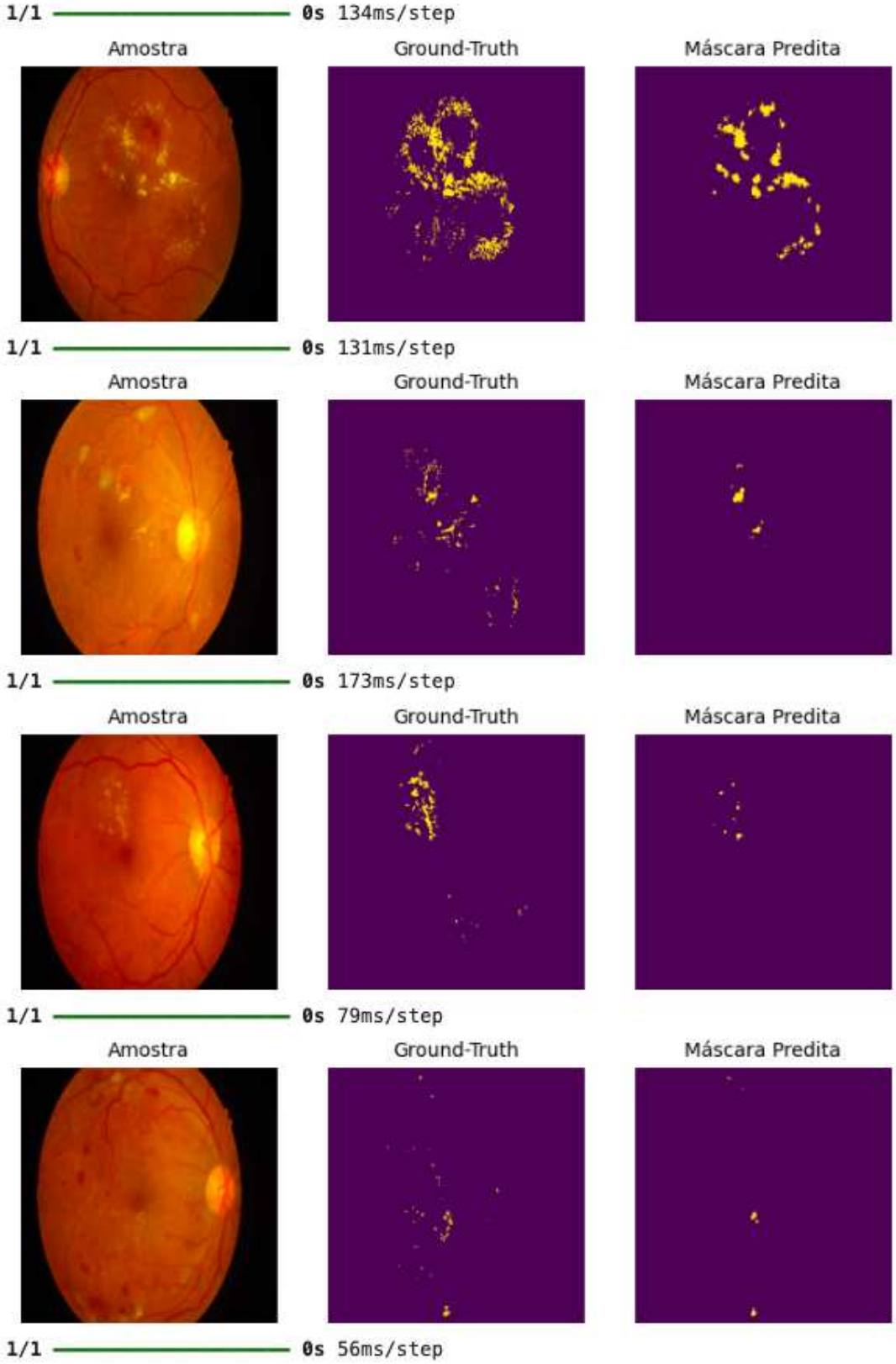
Fonte: Elaborado pelo autor.

Figura 17 – Focal + Dice Loss para o modelo U-Net



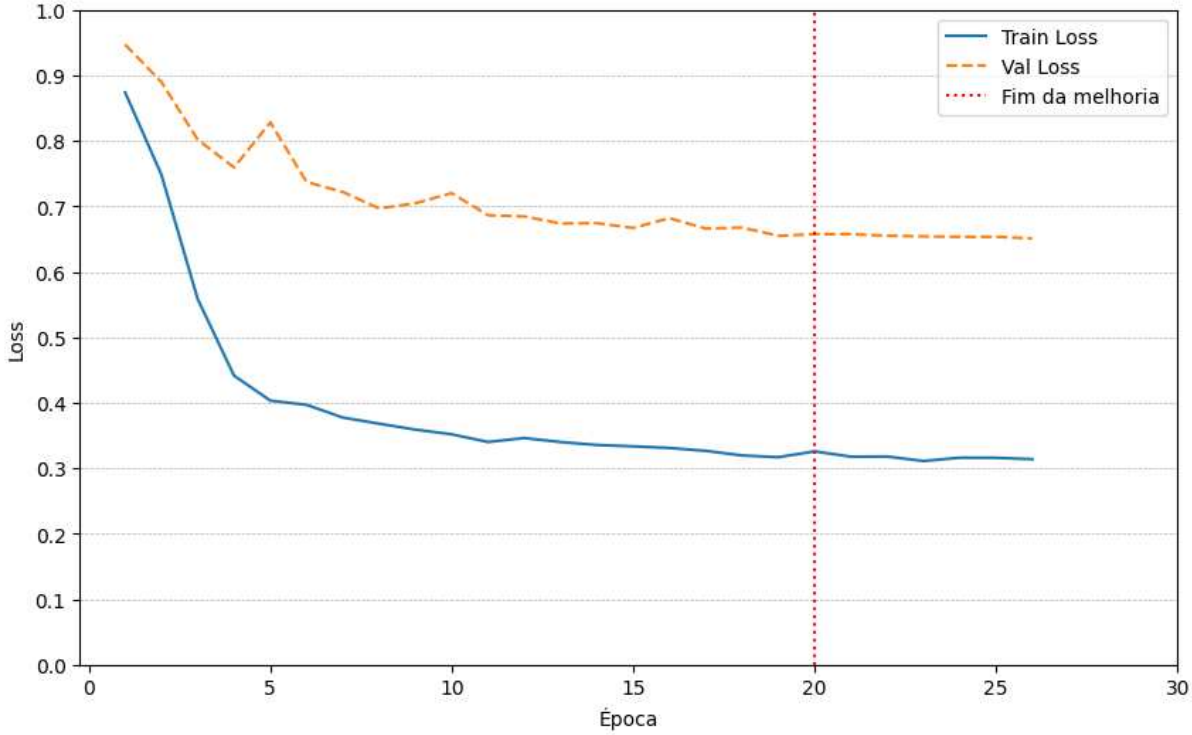
Fonte: Elaborado pelo autor.

Figura 18 – Amostra da segmentação semântica da U-Net



Fonte: Elaborado pelo autor.

Figura 19 – Tversky Loss no treino e teste da MobileViT



Fonte: Elaborado pelo autor.

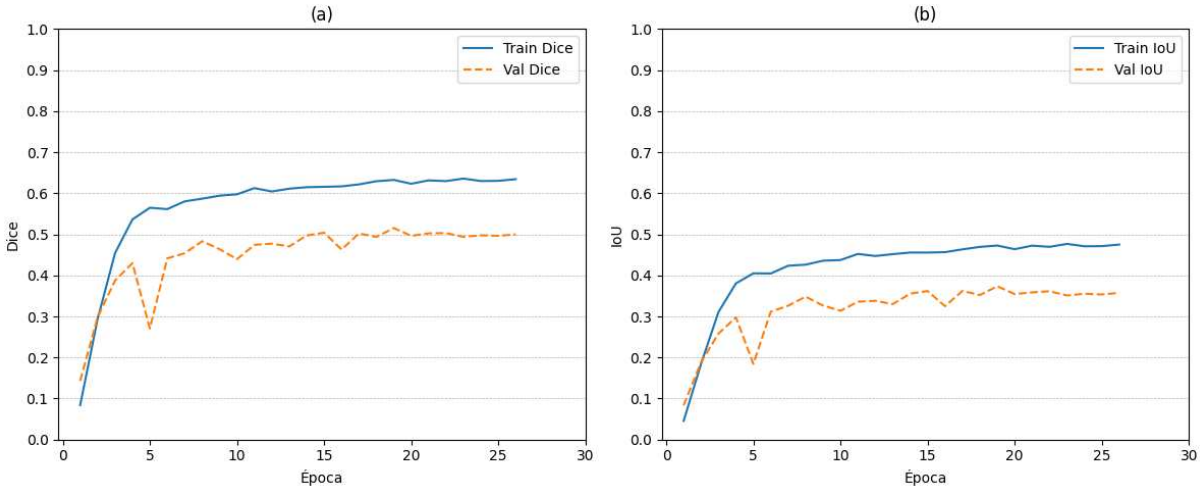
4.2 Treinamento da MobileViT

O desempenho do treinamento da MobileViT foram superiores ao da U-Net com MobileV2, embora as métricas demonstrem um valor abaixo do desejado para segmentação de imagens médicas. A partir dos hiperparâmetros adotados e explicados na Tabela 3, após 6 épocas, o modelo conseguiu atingir valores de Dice superiores aos do modelo anterior, por volta de 0,40. A Figura 19, mostra o decaimento da função de perda *tversky*. Como observado, a alta disparidade entre as funções de perda nos conjuntos de treino e teste do modelo, demonstram um *overfitting* inerente à modelagem. As curvas referentes às métricas IoU e coeficiente de Dice são significamente mais estáveis do que as da U-Net, indicando um regime de treinamento com menor variância.

4.3 Comparação dos Modelos

A Tabela 4 apresenta os resultados médios de Dice, IoU e tempo de inferência para o conjunto de imagens de teste, considerando o melhor modelo salvo. Ambos os modelos estudados desempenharam abaixo do esperado se comparado com modelos da literatura em

Figura 20 – Dice (a) e IoU (b) para 27 épocas de treinamento da MobileViT



Fonte: Elaborado pelo autor.

outras tarefas de segmentação semântica. Embora o coeficiente de Dice e IoU demonstrem um valor médio abaixo, observa-se que, na Figura 18, trouxeram os principais pixels que se destacam, no âmbito de contraste, na imagem de fundo de olho.

Tabela 4 – Comparativo de desempenho entre o MobileViT e a U-Net com MobileNetV2

Modelo	**Dice Score**	**IoU**	**Tempo de Inferência (s)**
MobileViT	**0.5152**	**0.3734**	**0.0015**
U-Net (MobileNetV2)	0.3452	0.2132	0.2491

Fonte: Elaborada pelo autor.

O MobileViT demonstrou uma superioridade significativa, não apenas na qualidade da segmentação, cerca de 49% superior ao modelo baseado em CNN, mas principalmente na eficiência computacional. O tempo de inferência de 0,0015 segundos sugere que o modelo é viável para aplicações em tempo real, representando um valor significativamente mais rápido que a U-Net.

5 CONCLUSÕES

O presente trabalho apresentou uma comparação entre um modelo de CNN e de *Vision Transformers* para a tarefa de segmentação semântica de exsudatos lipídicos (ou duros) em imagens de fundoscopia. Como demonstrado, embora seja um modelo promissor para identificação de objetos e reconhecimento de imagens, na segmentação semântica, para imagem com alto índice de desbalanceamento, a U-Net com MobileNetV2 como *encoder* apresentou um baixo desempenho na classificação semântica de pixels de exsudatos duros. A performance do MobileViT foi significantemente superior nas três métricas escolhidas. Este último, exibiu uma característica de *overfitting*, onde os resultados do conjunto de treinamento foram melhores que o modelo aplicado nas imagens de teste, demandando técnicas de *augmentation* e ajuste dos hiperparâmetros da arquitetura. Neste sentido, esse trabalho expôs o potencial da arquitetura *MobileViT* para a inferência em tempo real, que devem incentivar tecnologias de diagnóstico de doenças oculares baseadas em dispositivos de borda.

5.1 Limitações

As principais limitações deste trabalho estão associadas tanto ao ambiente computacional adotado quanto às características do conjunto de dados e da técnica de segmentação empregada. O desenvolvimento e o treinamento dos modelos foram realizados no ambiente Google Colab, o qual, embora disponibilize acesso gratuito a recursos de computação acelerada por GPU, impõe restrições quanto ao tempo de uso contínuo e à disponibilidade efetiva desses recursos. Tais limitações impactaram diretamente o número de épocas de treinamento, a realização de experimentos adicionais e a exploração mais aprofundada do espaço de hiperparâmetros.

No que se refere ao conjunto de dados, a quantidade reduzida de imagens disponíveis para treinamento e teste constitui uma limitação significativa. A escassez de amostras compromete a capacidade do modelo de aprender representações robustas e generalizáveis, especialmente em tarefas de segmentação semântica que envolvem estruturas pequenas, esparsas e altamente desbalanceadas em relação à classe de fundo, como os exsudatos lipídicos em imagens de retina.

Do ponto de vista metodológico, observou-se que o modelo apresenta melhor desempenho na segmentação de regiões sólidas e visualmente mais evidentes, enquanto demonstra limitações na identificação de áreas de menor contraste, regiões difusas ou parcialmente transpa-

rentes. Nessas situações, os pixels correspondentes às lesões tendem a ser classificados como fundo, o que resulta em falhas na segmentação e contribui para a redução das métricas de desempenho, como Dice e IoU. Esse comportamento evidencia a dificuldade do modelo em capturar padrões sutis em níveis mais profundos da imagem, especialmente sob a resolução adotada e diante das restrições impostas pelo treinamento. A disparidade entre as transformações na etapa de aumentação de dados dos dois modelos, também indica uma limitação quanto à análise comparativa proposta pelo trabalho.

Outra limitação relevante deste trabalho refere-se à redução da resolução das imagens para 256 por 256 pixels, realizada com o objetivo de viabilizar o treinamento dos modelos usando sua arquitetura base e devido às restrições computacionais impostas pelo ambiente de desenvolvimento adotado. Embora essa etapa seja comum em aplicações de aprendizado profundo, a diminuição da resolução espacial implica perda de detalhes finos e de informações de alta frequência, os quais são particularmente importantes para a segmentação de estruturas pequenas e pouco contrastantes, como os exsudatos lipídicos.

Por fim, tais limitações indicam que, embora a abordagem proposta apresente resultados promissores na identificação de lesões mais aparentes, melhorias futuras podem ser alcançadas por meio do aumento do conjunto de dados, do uso de estratégias de aumento de dados mais sofisticadas, do ajuste fino da arquitetura e da adoção de modelos explicitamente projetados para execução em ambientes com recursos computacionais limitados.

5.2 Trabalhos Futuros

Como continuidade deste trabalho e visando mitigar as limitações identificadas, recomenda-se, inicialmente, a adoção de estratégias mais robustas de data augmentation, especialmente para os modelos baseados em CNN. A incorporação de transformações geométricas adicionais, variações fotométricas e técnicas de aumento de dados mais avançadas pode contribuir para o aumento da variabilidade do conjunto de treinamento, favorecendo a generalização do modelo em cenários de maior complexidade visual.

Adicionalmente, sugere-se a realização de uma busca sistemática por hiperparâmetros por meio de métodos como grid search ou random search, contemplando parâmetros fundamentais do processo de otimização, tais como a taxa de aprendizado, o tipo de otimizador, o tamanho do lote (batch size) e os coeficientes associados às funções de perda. Tal abordagem pode resultar em configurações mais adequadas à natureza do problema de segmentação de

exsudatos lipídicos.

No que se refere à arquitetura do modelo, a exploração de diferentes redes de encoder na U-Net, como variantes baseadas em *EfficientNet* (BAHETI *et al.*, 2020), apresenta-se como uma alternativa promissora, uma vez que tais arquiteturas oferecem melhor equilíbrio entre capacidade representacional e eficiência computacional. Essa característica é particularmente relevante considerando aplicações futuras em ambientes com recursos computacionais restritos.

Outro aspecto relevante para trabalhos futuros consiste na incorporação de novos conjuntos de dados públicos e heterogêneos, possibilitando o aumento da diversidade amostral e a avaliação da capacidade de generalização dos modelos em diferentes condições de aquisição de imagem, resoluções e perfis populacionais. A utilização de múltiplos datasets pode reduzir o viés associado a uma única base de dados e contribuir para o desenvolvimento de modelos mais robustos e clinicamente relevantes.

Por fim, a implementação de técnicas de otimização de modelos, como *pruning* (poda) e *quantization* (quantização), pode contribuir para a redução da complexidade computacional e do consumo de memória, tornando o treinamento e a inferência mais eficientes. Essas abordagens são especialmente indicadas para viabilizar a implantação dos modelos em dispositivos embarcados ou sistemas portáteis de aquisição de imagens, aproximando a solução proposta de um cenário de aplicação prática.

REFERÊNCIAS

AZAD, R.; HEIDARY, M.; YILMAZ, K.; HüTTEMANN, M.; KARIMIJAFARBIGLOO, S.; WU, Y.; SCHMEINK, A.; MERHOF, D. Loss functions in the era of semantic segmentation: A survey and outlook. 12 2023. Disponível em: https://arxiv.org/pdf/2312.05391v1.

BAHETI, B.; INNANI, S.; GAJRE, S.; TALBAR, S. Eff-unet: A novel architecture for semantic segmentation in unstructured environment. **IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops**, IEEE Computer Society, p. 1473–1481, 6 2020. ISSN 21607516. Disponível em: https://ieeexplore.ieee.org/document/9150621.

BRASIL. **Portaria Conjunta SAES/SCTIE/MS nº 17, de 01 de outubro de 2021**. Ministério da Saúde, 2021. Portal CONITEC. Aprova o Protocolo Clínico e Diretrizes Terapêuticas da Retinopatia Diabética. Disponível em: https://www.gov.br/conitec/pt-br/midias/protocolos/20211220_portal_retinopatia_diabetica.pdf.

CISNEROS-GUZMáN, F.; TOVAR-ARRIAGA, S.; PEDRAZA, C.; GONZáLEZ-GUTIERREZ, A. Classification of diabetic retinopathy based on hard exudates patterns, using images processing and svm. **2019 IEEE Colombian Conference on Applications in Computational Intelligence, ColCACI 2019 - Proceedings**, Institute of Electrical and Electronics Engineers Inc., 6 2019. Disponível em: https://ieeexplore.ieee.org/document/8781802.

Data Science Academy. **Deep Learning Book**. 2025. Disponível em: https://www.deeplearningbook.com.br/. Acesso em: 07 jan. 2026.

DATACAMP. **Explicação das funções de perda no aprendizado de máquina**. 2024. Disponível em: https://www.datacamp.com/pt/tutorial/loss-function-in-machine-learning. Acesso em: 08 jan. 2026.

DOSOVITSKIY, A.; BEYER, L.; KOLESNIKOV, A.; WEISSENBORN, D.; ZHAI, X.; UNTERTHINER, T.; DEHGHANI, M.; MINDERER, M.; HEIGOLD, G.; GELLY, S.; USZKOREIT, J.; HOULSBY, N. An image is worth 16x16 words: Transformers for image recognition at scale. **ICLR 2021 - 9th International Conference on Learning Representations**, International Conference on Learning Representations, ICLR, 10 2020. Disponível em: https://arxiv.org/pdf/2010.11929.

ESTEVAM, R. **Redes Neurais Convolucionais**. 2019. Disponível em: https://medium.com/turing-talks/turing-talks-23-modelos-de-predi%C3%A7%C3%A3o-redes-neurais-convolucionais-d364654a34de.

GROUP, E. R. Grading diabetic retinopathy from stereoscopic color fundus photographs—an extension of the modified airlie house classification: Etdrs report number 10. **Ophthalmology**, v. 98, p. 786–806, 5 1991. Disponível em: https://www.sciencedirect.com/science/article/pii/S0161642013380129.

IBM. **O que são redes neurais convolucionais?** Disponível em: https://www.ibm.com/br-pt/think/topics/convolutional-neural-networks.

LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, v. 86, p. 2278–2324, 11 1998. ISSN 00189219. Disponível em: https://ieeexplore.ieee.org/document/726791.

MATIAS, A. V.; AMORIM, J. G. A.; MACARINI, L. A. B.; CERENTINI, A.; ONOFRE, A. S. C.; ONOFRE, F. B. D. M.; DALTOé, F. P.; STEMMER, M. R.; WANGENHEIM, A. von. What is the state of the art of computer vision-assisted cytology? a systematic literature review. **Computerized Medical Imaging and Graphics**, Pergamon, v. 91, p. 101934, 7 2021. ISSN 0895-6111. Disponível em: https://www.sciencedirect.com/science/article/abs/pii/S0895611121000835?via%3Dihub.

MEHTA, S.; RASTEGARI, M. Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer. **ICLR 2022 - 10th International Conference on Learning Representations**, International Conference on Learning Representations, ICLR, 10 2021. Disponível em: https://arxiv.org/pdf/2110.02178.

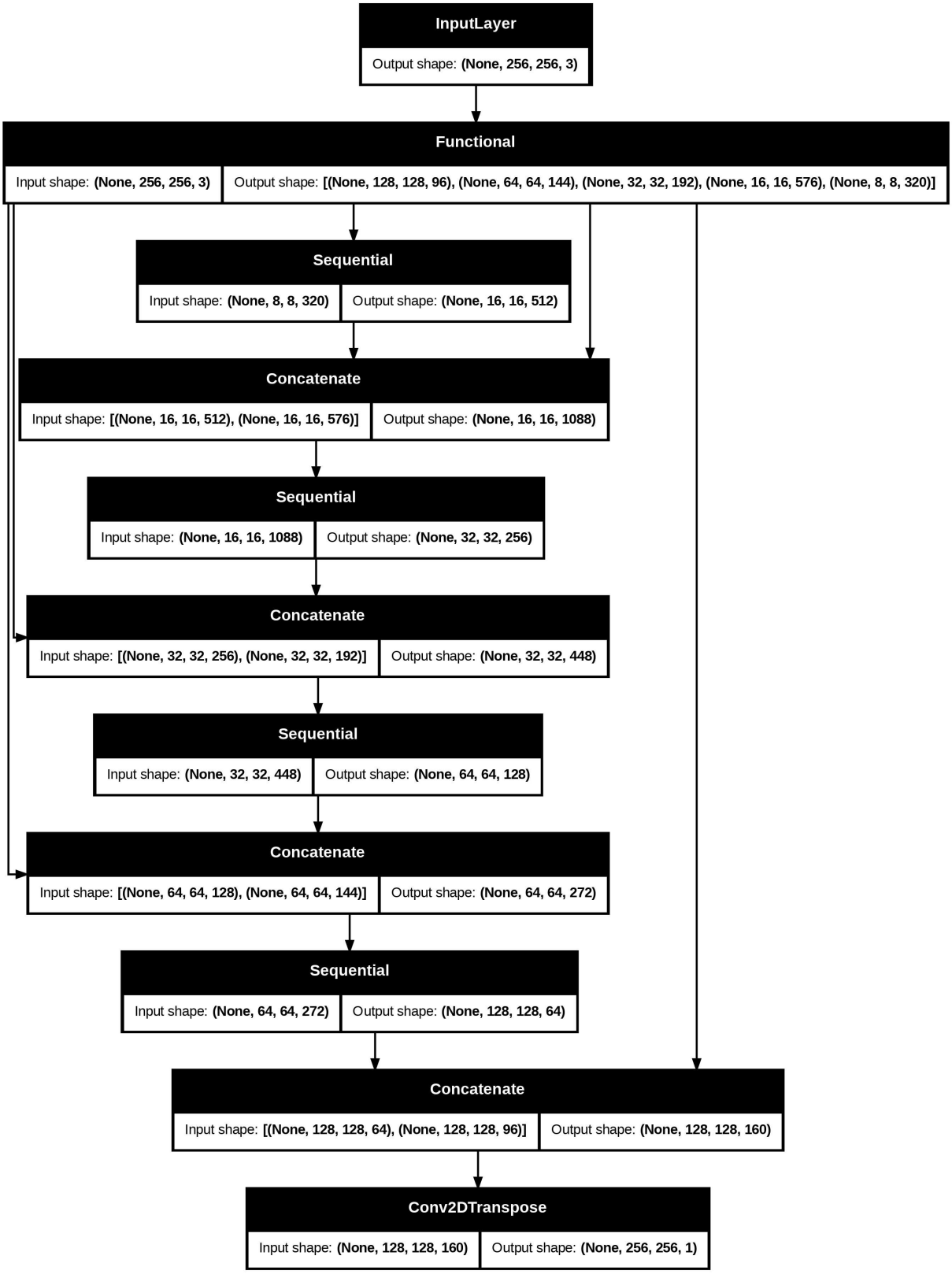
PORWAL, P.; PACHADE, S.; KAMBLE, R.; KOKARE, M.; DESHMUKH, G.; SAHASRABUDDHE, V.; MERIAUDEAU, F. **Indian Diabetic Retinopathy Image Dataset (IDRiD)**. IEEE Dataport, 2018. Disponível em: https://dx.doi.org/10.21227/H25W98.

PRAYOGO, M. E.; ZAHARO, A. F.; DAMAYANTI, N. N. R.; WIDYAPUTRI, F.; THOBARI, J. A.; SUSANTI, V. Y.; SASONGKO, M. B. Accuracy of low-cost, smartphone-based retinal photography for diabetic retinopathy screening: A systematic review. **Clinical Ophthalmology (Auckland, N.Z.)**, Dove Medical Press Ltd, v. 17, p. 2459–2470, 8 2023. ISSN 11775483. Disponível em: https://pmc.ncbi.nlm.nih.gov/articles/PMC10443682/.

RONNEBERGER, O.; FISCHER, P.; BROX, T. U-net: Convolutional networks for biomedical image segmentation. 5 2015. Disponível em: http://arxiv.org/abs/1505.04597.

ROSENBLATT, F. The perceptron: A probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, 11 1958.

APÊNDICE A – ARQUITETURA U-NET COM MOBILENETV2



APÊNDICE B – CÓDIGO-FONTE DA ARQUITETURA U-NET MODELADA

```
# -*- coding: utf-8 -*-
"""Lumina_CNNs.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1hQKFGFdqdnugVfuvkaI0XM2WsAjd18Sy

---

<center><h3></h3><h3><b>Comparação entre Vision Transformers e U-Net com
    MobileNetV2 na Segmentação de Exsudatos Lipídicos em Imagens de
    Retinografia

</b></h3></center>

Este notebook apresenta a análise das metodologias de segmentação de imagens
    de fundo de olho no diagnóstico de retinopatia diabética, como resultado
    do projeto final do Trabalho de Conclusão de Curso em Engenharia Elétrica
    pela Universidade Federal do Ceará (UFC).

Autor: Pedro Florencio

---

<!-- Python -->
<a href="https://www.python.org/downloads/release/python-3100/" style="margin
    -right:12px;">
  <img align="left" src="https://img.shields.io/badge/python-3.10-green" alt
    ="Python Version" title="Python Version" height="20">
</a>

<!-- TensorFlow -->
<a href="https://www.tensorflow.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/TensorFlow-2.x-FF6F00?
    logo=tensorflow&logoColor=white" alt="TensorFlow" title="TensorFlow"
    height="20">
</a>

<!-- OpenCV -->
<a href="https://opencv.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/OpenCV-4.x-blue?logo=
    opencv&logoColor=white" alt="OpenCV" title="OpenCV" height="20">
</a>
```

```
<!-- scikit-image -->
<a href="https://scikit-image.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/scikit--image-0.21-orange?logo=scikit-learn&logoColor=white" alt="scikit-image" title="scikit-image" height="20">
</a>

<!-- Licença Creative Commons -->
<a href="https://creativecommons.org/licenses/by-nd/4.0/" target="_blank" style="margin-right:12px;">
  <img align="left" src="https://licensebuttons.net/l/by-nd/4.0/88x31.png" alt="Creative Commons BY-ND 4.0" title="Creative Commons BY-ND 4.0" height="20">
</a>
"""

# GPU
!nvidia-smi

# CPU
!cat /proc/cpuinfo | grep "model name" | uniq

# Memória RAM
!cat /proc/meminfo | grep MemTotal

""#### **1. Bibliotecas**

"""

!pip install -q Pillow
!pip install -q tensorflow
!pip install -q opencv-python
!pip install -q git+https://github.com/tensorflow/examples.git

import os
import glob
import time
import keras
import cv2 as cv
import numpy as np
import pandas as pd
from PIL import Image
import tensorflow as tf
from google.colab import drive
import matplotlib.pyplot as plt
from IPython.display import clear_output
```

```
77 from tensorflow_examples.models.pix2pix import pix2pix
78
79 drive.mount('/gdrive', force_remount=True)
80
81 """#### **2. Aquisição das Imagens**
82
83 Etapas no processo de aquisição da imagem:
84 - Passo 1: Leitura do arquivo da imagem;
85 - Passo 2: Decodificação da imagem em 3 canais para construção de uma matriz;
86 - Passo 3: Aplicando intervalo de 0 a 1;
87 - Passo 4: Redimensionamento da imagem;
88 - Passo 5: Leitura da máscara;
89 - Passo 6: Criação da matriz com 1 canal
90 - Passo 7: Redimensionamento da máscara com método Nearest
91 - Passo 8: Binarização da máscara
92
93 **Constantes**
94 """
95
96 IMG_SIZE = 256
97 BATCH_SIZE = 4
98 FOLDER = '/gdrive/MyDrive/Lumina/'
99
100 """##### **2.1. Conversão das máscaras de .TIF para .PNG**"""
101
102 caminho_tif_train = FOLDER + 'Data/A. Segmentation/2. All Segmentation
    Groundtruths/a. Training Set/3. Hard Exudates/'
103 arquivos_tif_train = glob.glob(os.path.join(caminho_tif_train, "*.tif"))
104 caminho_tif_test = FOLDER + 'Data/A. Segmentation/2. All Segmentation
    Groundtruths/b. Testing Set/3. Hard Exudates/'
105 arquivos_tif_test = glob.glob(os.path.join(caminho_tif_test, "*.tif"))
106
107 print(f"Encontradas {len(arquivos_tif_train)} máscaras de treino e {len(
    arquivos_tif_test)} máscaras de teste .tif para converter.")
108
109 for arquivo in arquivos_tif_train:
110     try:
111         img = Image.open(arquivo)
112         novo_nome = arquivo.replace(".tif", ".png")
113         img.save(novo_nome, "PNG")
114         # os.remove(arquivo)
115
116     except Exception as e:
117         print(f"Erro ao converter {arquivo}: {e}")
118
119 for arquivo in arquivos_tif_test:
120     try:
```

```
        img = Image.open(arquivo)
        novo_nome = arquivo.replace(".tif", ".png")
        img.save(novo_nome, "PNG")
        # os.remove(arquivo)

    except Exception as e:
        print(f"Erro ao converter {arquivo}: {e}")

print("Conversão concluída!")

"""##### **2.2. Listas de caminhos dos arquivos**

<mark>Treino
"""

train_img_paths = sorted(glob.glob(FOLDER + 'Data/A. Segmentation/1. Original Images/a. Training Set/*.jpg'))
train_mask_paths = sorted(glob.glob(FOLDER + 'Data/A. Segmentation/2. All Segmentation Groundtruths/a. Training Set/3. Hard Exudates/*.png'))

print(f"Imagens encontradas: {len(train_img_paths)}")
print(f"Máscaras encontradas: {len(train_mask_paths)}")

"""<mark>Teste"""

test_img_paths = sorted(glob.glob(FOLDER + 'Data/A. Segmentation/1. Original Images/b. Testing Set/*.jpg'))
test_mask_paths = sorted(glob.glob(FOLDER + 'Data/A. Segmentation/2. All Segmentation Groundtruths/b. Testing Set/3. Hard Exudates/*.png'))

print(f"Imagens encontradas: {len(test_img_paths)}")
print(f"Máscaras encontradas: {len(test_mask_paths)}")

"""##### **2.3. Carregamento e processamento das imagens**"""

def load_and_preprocess(img_path, mask_path):

    # carregamento e processamento da imagem
    img = tf.io.read_file(img_path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE])
    img = tf.cast(img, tf.float32) / 255.0

    # carregamento e processamento da mascara
    mask = tf.io.read_file(mask_path)
    mask = tf.image.decode_png(mask, channels=1)
    mask = tf.image.resize(mask, [IMG_SIZE, IMG_SIZE], method='nearest')
```

```
    mask = tf.cast(mask > 0, tf.int32)

    return img, mask

"""##### **2.4. Formato de Tensorflow Dataset**"""

dataset_train = tf.data.Dataset.from_tensor_slices(
    (train_img_paths, train_mask_paths)
)

dataset_train = dataset_train.map(
    load_and_preprocess,
    num_parallel_calls=tf.data.AUTOTUNE
)

dataset_train = dataset_train.shuffle(1000)
dataset_train = dataset_train.batch(BATCH_SIZE)
dataset_train = dataset_train.prefetch(tf.data.AUTOTUNE)

dataset_test = tf.data.Dataset.from_tensor_slices(
    (test_img_paths, test_mask_paths)
)

dataset_test = dataset_test.map(
    load_and_preprocess,
    num_parallel_calls=tf.data.AUTOTUNE
)

dataset_test = dataset_test.batch(BATCH_SIZE)
dataset_test = dataset_test.prefetch(tf.data.AUTOTUNE)

"""#### **3. Visualização das Imagens**

Visualizando de imagem exemplo de fundo de olho do dataset
"""

FIGSIZE = (8,8)

def display(display_list, main_title=None):
  plt.figure(figsize=FIGSIZE)

  title = ['Amostra', 'Ground-Truth', 'Máscara Predita']

  for i in range(len(display_list)):
    plt.subplot(1,len(display_list),i+1)
    plt.title(title[i],{'fontsize':10})
    plt.imshow(tf.keras.preprocessing.image.array_to_img(display_list[i]))
```

```
    plt.axis('off')

  plt.show()

for image, mask in dataset_test.take(1):
    sample_image, sample_mask = image[0], mask[0]
    display([sample_image, sample_mask])

"""#### **4. Data Augmentation**

##### **4.1. Inversão das imagens**
"""

@tf.function
def load_image_train(input_image, input_mask):
  input_image = tf.image.resize(input_image, (IMG_SIZE, IMG_SIZE))
  input_mask = tf.image.resize(input_mask, (IMG_SIZE, IMG_SIZE), method='
    nearest')

  if tf.random.uniform(()) > 0.5:
    input_image = tf.image.flip_left_right(input_image)
    input_mask = tf.image.flip_left_right(input_mask)

  return input_image, input_mask

def load_image_test(input_image, input_mask):
  input_image = tf.image.resize(input_image, (IMG_SIZE, IMG_SIZE))
  input_mask = tf.image.resize(input_mask, (IMG_SIZE, IMG_SIZE))

  return input_image, input_mask

TRAIN_LENGTH = len(train_img_paths)
BUFFER_SIZE = TRAIN_LENGTH
STEPS_PER_EPOCH = TRAIN_LENGTH // BATCH_SIZE

"""#### **5. Definição de treino e teste**"""

train = dataset_train.map(load_image_train, num_parallel_calls=tf.data.
    experimental.AUTOTUNE)
test = dataset_test.map(load_image_test)

train_dataset = train
test_dataset = test

print("Verificando o shape de um batch:")
for img, mask in train_dataset.take(1):
    print(f"Shape esperado: (4, 128, 128, 3)")
```

```
    print(f"Shape real:     {img.shape}")

"""#### **6. Modelagem**

##### **6.1. Definição da UNet Modificada**
"""

# quantidade de canais (ou classes) de saida
OUTPUT_CHANNELS = 1

base_model = tf.keras.applications.MobileNetV2(input_shape=[IMG_SIZE,
    IMG_SIZE, 3], include_top=False)

# Use as ativações dessas camadas
layer_names = [
    'block_1_expand_relu',   # 64x64
    'block_3_expand_relu',   # 32x32
    'block_6_expand_relu',   # 16x16
    'block_13_expand_relu',  # 8x8
    'block_16_project',      # 4x4
]
layers = [base_model.get_layer(name).output for name in layer_names]

# Crie o modelo de extração de características
down_stack = tf.keras.Model(inputs=base_model.input, outputs=layers)

for layer in down_stack.layers[-30:]:
    layer.trainable = True

up_stack = [
    pix2pix.upsample(512, 3),  # 4x4 -> 8x8
    pix2pix.upsample(256, 3),  # 8x8 -> 16x16
    pix2pix.upsample(128, 3),  # 16x16 -> 32x32
    pix2pix.upsample(64, 3),   # 32x32 -> 64x64
]

def unet_model(output_channels):

  last = tf.keras.layers.Conv2DTranspose(
      output_channels, 3, strides=2,
      padding='same', activation='sigmoid')

  inputs = tf.keras.layers.Input(shape=[IMG_SIZE, IMG_SIZE, 3])
  x = inputs

  skips = down_stack(x)
  x = skips[-1]
```

```
  skips = reversed(skips[:-1])

  for up, skip in zip(up_stack, skips):
    x = up(x)
    concat = tf.keras.layers.Concatenate()
    x = concat([x, skip])

  x = last(x)

  return tf.keras.Model(inputs=inputs, outputs=x)

"""##### **6.2. Métricas de avaliação**"""

def dice_loss(y_true, y_pred, smooth=1e-6):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred, tf.float32)

    intersection = tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred)

    return 1 - (2. * intersection + smooth) / (union + smooth)

def bce_dice_loss(y_true, y_pred):
    bce = tf.keras.losses.BinaryCrossentropy()(y_true, y_pred)
    return bce + dice_loss(y_true, y_pred)

def dice_coef(y_true, y_pred, smooth=1e-6):
    y_pred = tf.cast(y_pred > 0.5, tf.float32)
    y_true = tf.cast(y_true, tf.float32)

    intersection = tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred)

    return (2. * intersection + smooth) / (union + smooth)

def iou_metric(y_true, y_pred, smooth=1e-6):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.cast(y_pred > 0.5, tf.float32)

    intersection = tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true + y_pred) - intersection

    return (intersection + smooth) / (union + smooth)

def focal_loss(y_true, y_pred, alpha=0.8, gamma=2):
    y_true = tf.cast(y_true, tf.float32)
    y_pred = tf.clip_by_value(y_pred, 1e-7, 1-1e-7)
```

```
349
350     pt = tf.where(tf.equal(y_true, 1), y_pred, 1-y_pred)
351     return -tf.reduce_mean(alpha * tf.pow(1-pt, gamma) * tf.math.log(pt))
352
353 def focal_dice_loss(y_true, y_pred):
354     return focal_loss(y_true, y_pred) + dice_loss(y_true, y_pred)
355
356 """##### **6.3. Treinamento**"""
357
358 model = unet_model(OUTPUT_CHANNELS)
359 model.compile(
360     optimizer=tf.keras.optimizers.Adam(1e-4),
361     loss=focal_dice_loss,
362     metrics=[dice_coef, iou_metric]
363 )
364
365 tf.keras.utils.plot_model(model, show_shapes=True)
366
367 """##### **6.3. Visualização antes do treinamento**"""
368
369 def create_mask(pred_mask):
370     pred_mask = (pred_mask > 0.5)
371     return pred_mask[0]
372
373 def show_predictions(dataset=None, num=1):
374   if dataset:
375     for image, mask in dataset.take(num):
376       pred_mask = model.predict(image)
377       display([image[0], mask[0], create_mask(pred_mask)])
378   else:
379     display([sample_image, sample_mask,
380              create_mask(model.predict(sample_image[tf.newaxis, ...]))])
381
382 show_predictions()
383
384 """#### **7. Resultados**
385
386 ##### **7.1. Visualização**
387 """
388
389 class DisplayCallback(tf.keras.callbacks.Callback):
390   def on_epoch_end(self, epoch, logs=None):
391     clear_output(wait=True)
392     show_predictions()
393     print ('\nSample Prediction after epoch {}\n'.format(epoch+1))
394
395 EPOCHS = 100
```

```
VALIDATION_STEPS = None

for img, mask in train_dataset.take(1):
    pred = model(img)
    print(pred.shape)
    print(mask.shape)
    print(pred.dtype, mask.dtype)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',        # observa a perda de validação
    patience=5,               # quantas épocas esperar sem melhora
    restore_best_weights=True # volta para o melhor modelo
)

# Commented out IPython magic to ensure Python compatibility.
# %%time
# model_history = model.fit(train_dataset,
#                           epochs=EPOCHS,
#                           steps_per_epoch=STEPS_PER_EPOCH,
#                           validation_steps=VALIDATION_STEPS,
#                           validation_data=test_dataset,
#                           callbacks=[DisplayCallback(),early_stopping])

loss = model_history.history['loss']
val_loss = model_history.history['val_loss']

# Ajusta o número de épocas para o número real de épocas treinadas
epochs = range(len(loss))

plt.figure()
plt.plot(epochs, loss, 'g', label='Função de Perda - Treino')
plt.plot(epochs, val_loss, 'purple', label='Função de Perda - Validação')
plt.title('')
plt.yticks(np.arange(0, 1.51, 0.1))
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xlabel(' poca ')
plt.ylabel('Loss Value')
plt.legend()
plt.show()

dice = model_history.history['dice_coef']
val_dice = model_history.history['val_dice_coef']
iou = model_history.history['iou_metric']
val_iou = model_history.history['val_iou_metric']

epochs = range(len(dice))

```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs, dice, 'r', label='Training Dice Coef')
plt.plot(epochs, val_dice, 'bo', label='Validation Dice Coef')
plt.title('(a)')
plt.xlabel(' poca ')
plt.ylabel('Dice')
plt.yticks(np.arange(0, 1.1, 0.1))
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(epochs, iou, 'r', label='Training IoU Metric')
plt.plot(epochs, val_iou, 'bo', label='Validation IoU Metric')
plt.title('(b)')
plt.yticks(np.arange(0, 1.1, 0.1))
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.xlabel(' poca ')
plt.ylabel('IoU')
plt.legend()

plt.show()

show_predictions(test_dataset, 7)

"""##### **7.2. Avaliação**"""

def dice_score(y_true, y_pred, smooth=1e-6):
    y_pred = tf.cast(y_pred > 0.5, tf.float32)
    intersection = tf.reduce_sum(y_true * y_pred)
    return (2. * intersection + smooth) / (
        tf.reduce_sum(y_true) + tf.reduce_sum(y_pred) + smooth
    )


def iou_score(y_true, y_pred, smooth=1e-6):
    y_pred = tf.cast(y_pred > 0.5, tf.float32)
    intersection = tf.reduce_sum(y_true * y_pred)
    union = tf.reduce_sum(y_true) + tf.reduce_sum(y_pred) - intersection
    return (intersection + smooth) / (union + smooth)

def pixel_accuracy(y_true, y_pred):
    y_pred = tf.cast(y_pred > 0.5, tf.float32)
    correct = tf.equal(y_true, y_pred)
    return tf.reduce_mean(tf.cast(correct, tf.float32))

```

```
490 def evaluate_segmentation_metrics(model, dataset):
491     dice_list = []
492     iou_list = []
493     acc_list = []
494 
495     for x, y_true in dataset:
496         y_pred = model(x, training=False)
497 
498         dice_list.append(dice_score(y_true, y_pred).numpy())
499         iou_list.append(iou_score(y_true, y_pred).numpy())
500         acc_list.append(pixel_accuracy(y_true, y_pred).numpy())
501 
502     return {
503         "dice_mean": np.mean(dice_list),
504         "dice_std": np.std(dice_list),
505         "iou_mean": np.mean(iou_list),
506         "iou_std": np.std(iou_list),
507         "acc_mean": np.mean(acc_list),
508         "acc_std": np.std(acc_list),
509     }
510 
511 def measure_inference_time(model, dataset, n_batches=20):
512     times = []
513 
514     for i, (x, _) in enumerate(dataset.take(n_batches)):
515         start = time.time()
516         _ = model(x, training=False)
517         times.append(time.time() - start)
518 
519     return {
520         "time_mean": np.mean(times),
521         "time_std": np.std(times)
522     }
523 
524 def estimate_energy_consumption(
525     inference_time_sec,
526     avg_power_watts=50  # ajuste conforme CPU/GPU
527 ):
528     # Energia = Potência    Tempo
529     energy_joules = avg_power_watts * inference_time_sec
530     return energy_joules
531 
532 def full_model_analysis(model, dataset, power_watts=50):
533     metrics = evaluate_segmentation_metrics(model, dataset)
534     timing = measure_inference_time(model, dataset)
535 
536     energy = estimate_energy_consumption(
```

```
        timing["time_mean"],
        avg_power_watts=power_watts
    )

    return {
        "Dice": f"{metrics['dice_mean']:.4f}    {metrics['dice_std']:.4f}",
        "IoU": f"{metrics['iou_mean']:.4f}    {metrics['iou_std']:.4f}",
        "Accuracy": f"{metrics['acc_mean']:.4f}    {metrics['acc_std']:.4f}",
        "Inference time (s)": f"{timing['time_mean']:.4f}    {timing['
    time_std']:.4f}",
        "Energy (J)": f"{energy:.2f}"
    }

results = full_model_analysis(
    model,
    test_dataset,
    power_watts=50  # ajuste conforme sua máquina
)

for k, v in results.items():
    print(f"{k}: {v}")

"""**Deus é bom o tempo todo!**"""
```

APÊNDICE C – CÓDIGO-FONTE DA ARQUITETURA MOBILEVIT MODELADA

```
# -*- coding: utf-8 -*-
"""Lumina_ViTs.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1Gw0RQGdtLGXeUbs7Ezw2fo25lGviPszO

---

<center><h3></h3><h3><b>Comparação entre Vision Transformers e U-Net com
    MobileNetV2 na Segmentação de Exsudatos Lipídicos em Imagens de
    Retinografia

</b></h3></center>

Este notebook apresenta a análise das metodologias de segmentação de imagens
    de fundo de olho no diagnóstico de retinopatia diabética, como resultado
    do projeto final do Trabalho de Conclusão de Curso em Engenharia Elétrica
    pela Universidade Federal do Ceará (UFC).

Autor: Pedro Florencio
<br>Contato: <i>pedroflorencio@edu.unifor.br</i>

---

<!-- Python -->
<a href="https://www.python.org/downloads/release/python-3100/" style="margin
    -right:12px;">
  <img align="left" src="https://img.shields.io/badge/python-3.10-green" alt
    ="Python Version" title="Python Version" height="20">
</a>

<!-- TensorFlow -->
<a href="https://www.tensorflow.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/TensorFlow-2.x-FF6F00?
    logo=tensorflow&logoColor=white" alt="TensorFlow" title="TensorFlow"
    height="20">
</a>

<!-- OpenCV -->
<a href="https://opencv.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/OpenCV-4.x-blue?logo=
    opencv&logoColor=white" alt="OpenCV" title="OpenCV" height="20">
```

```
</a>

<!-- scikit-image -->
<a href="https://scikit-image.org/" style="margin-right:12px;">
  <img align="left" src="https://img.shields.io/badge/scikit--image-0.21-orange?logo=scikit-learn&logoColor=white" alt="scikit-image" title="scikit-image" height="20">
</a>

<!-- Licença Creative Commons -->
<a href="https://creativecommons.org/licenses/by-nd/4.0/" target="_blank" style="margin-right:12px;">
  <img align="left" src="https://licensebuttons.net/l/by-nd/4.0/88x31.png" alt="Creative Commons BY-ND 4.0" title="Creative Commons BY-ND 4.0" height="20">
</a>
"""

# GPU
!nvidia-smi

# CPU
!cat /proc/cpuinfo | grep "model name" | uniq

# Memória RAM
!cat /proc/meminfo | grep MemTotal

""#### **1. Bibliotecas**

"""

!pip install -q torch torchvision transformers datasets pillow scikit-learn numpy

# Commented out IPython magic to ensure Python compatibility.
import os
import cv2
import torch
import numpy as np
from glob import glob
from tqdm import tqdm
from PIL import Image
import torch.nn as nn
import seaborn as sns
import albumentations as A
from torch.optim import AdamW
import torch.nn.functional as F
```

```
import matplotlib.pyplot as plt
from google.colab import userdata
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.optim.lr_scheduler import CosineAnnealingLR
from transformers import MobileViTForSemanticSegmentation,
    MobileViTImageProcessor

# %matplotlib inline

from google.colab import drive
drive.mount('/gdrive', force_remount=True)

"""#### **2. Par metros**"""

BASE_DRIVE = '/gdrive/MyDrive/Lumina/Data/A. Segmentation'

PATHS = {
    'train': {
        'img': '/gdrive/MyDrive/Lumina/Data/A. Segmentation/1. Original
    Images/a. Training Set/',
        'mask': '/gdrive/MyDrive/Lumina/Data/A. Segmentation/2. All
    Segmentation Groundtruths/a. Training Set/3. Hard Exudates/'
    },
    'test': {
        'img': '/gdrive/MyDrive/Lumina/Data/A. Segmentation/1. Original
    Images/b. Testing Set/',
        'mask': '/gdrive/MyDrive/Lumina/Data/A. Segmentation/2. All
    Segmentation Groundtruths/b. Testing Set/3. Hard Exudates/'
    }
}

PATCH_SIZE = 256
STRIDE = 256
OUTPUT_DIR = "/content/IDRiD_Patches"
HF_TOKEN = userdata.get('HF_TOKEN')
MODEL_CHECKPOINT = "apple/mobilevit-small"
BATCH_SIZE = 8
EPOCHS = 30
LEARNING_RATE = 1e-4

"""#### **3. Pré-Processamento das Imagens**"""

def process_dataset(split_type):
    print(f"--- Processando {split_type.upper()} ---")

    img_dir = PATHS[split_type]['img']
```

```
    mask_dir = PATHS[split_type]['mask']

    # Listar todas as imagens JPG
    image_files = sorted(glob(os.path.join(img_dir, "*.jpg")))

    save_img_dir = os.path.join(OUTPUT_DIR, split_type, "images")
    save_mask_dir = os.path.join(OUTPUT_DIR, split_type, "masks")
    os.makedirs(save_img_dir, exist_ok=True)
    os.makedirs(save_mask_dir, exist_ok=True)

    count_saved = 0

    for img_path in tqdm(image_files):
        filename = os.path.basename(img_path)
        name_no_ext = os.path.splitext(filename)[0]
        mask_name_tif = f"{name_no_ext}_EX.tif"
        mask_name_png = f"{name_no_ext}_EX.png"

        full_mask_path = os.path.join(mask_dir, mask_name_tif)

        if not os.path.exists(full_mask_path):
            # Tenta opção PNG se TIF não existir
            full_mask_path = os.path.join(mask_dir, mask_name_png)
            if not os.path.exists(full_mask_path):
                print(f"AVISO: Máscara não encontrada para {filename}. Pulando.")
                continue



        img = cv2.imread(img_path)
        mask = cv2.imread(full_mask_path, 0)

        if img is None or mask is None:
            continue

        mask = np.where(mask > 0, 1, 0).astype(np.uint8)

        h, w, _ = img.shape

        idx_patch = 0
        for y in range(0, h - PATCH_SIZE + 1, STRIDE):
            for x in range(0, w - PATCH_SIZE + 1, STRIDE):
                patch_img = img[y:y+PATCH_SIZE, x:x+PATCH_SIZE]
                patch_mask = mask[y:y+PATCH_SIZE, x:x+PATCH_SIZE]
                has_exudates = np.sum(patch_mask) > 0

```

```
                keep_patch = False
                if split_type == 'test':
                    keep_patch = True
                else:
                    if has_exudates:
                        keep_patch = True
                    elif np.random.rand() < 0.05:
                        keep_patch = True

                if keep_patch:
                    save_name = f"{name_no_ext}_{idx_patch}.png"
                    cv2.imwrite(os.path.join(save_img_dir, save_name), patch_img)
                    cv2.imwrite(os.path.join(save_mask_dir, save_name), patch_mask)
                    count_saved += 1

                idx_patch += 1

    print(f"Total de patches gerados para {split_type}: {count_saved}")

process_dataset('train')
process_dataset('test')

train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.5),
    A.RandomRotate90(p=0.5),
    A.ShiftScaleRotate(shift_limit=0.0625, scale_limit=0.1, rotate_limit=45, p=0.5),
    A.RandomBrightnessContrast(p=0.2),
    A.GridDistortion(p=0.2),
])

"""#### **4. Modelagem do Dataset**"""

class IDRIDPatchDataset(Dataset):
    def __init__(self, root_dir, split, processor, transform=None):
        self.img_dir = os.path.join(root_dir, split, "images")
        self.mask_dir = os.path.join(root_dir, split, "masks")

        self.files = sorted(os.listdir(self.img_dir))
        self.processor = processor
        self.transform = transform

    def __len__(self):
        return len(self.files)
```

```
208
209     def __getitem__(self, idx):
210         file_name = self.files[idx]
211         img_path = os.path.join(self.img_dir, file_name)
212         mask_path = os.path.join(self.mask_dir, file_name)
213
214         image = Image.open(img_path).convert("RGB")
215         mask = Image.open(mask_path).convert("L")
216
217         image_np = np.array(image)
218         mask_np = np.array(mask)
219
220         if self.transform:
221             augmented = self.transform(image=image_np, mask=mask_np)
222             image_np = augmented['image']
223             mask_np = augmented['mask']
224
225         inputs = self.processor(
226             images=image_np,
227             segmentation_maps=mask_np,
228             return_tensors="pt"
229         )
230
231         inputs = {k: v.squeeze(0) for k, v in inputs.items()}
232
233         return inputs
234
235 processor = MobileViTImageProcessor.from_pretrained(MODEL_CHECKPOINT)
236 train_ds = IDRIDPatchDataset(OUTPUT_DIR, 'train', processor, transform=
    train_transform)
237 test_ds = IDRIDPatchDataset(OUTPUT_DIR, 'test', processor, transform=None) #
    Sem augmentação no teste!
238
239 class DiceLoss(nn.Module):
240     def __init__(self, smooth=1):
241         super(DiceLoss, self).__init__()
242         self.smooth = smooth
243
244     def forward(self, inputs, targets):
245
246         inputs = F.softmax(inputs, dim=1)
247         inputs = inputs[:, 1, :, :].contiguous().view(-1)
248         targets = targets.contiguous().view(-1)
249         intersection = (inputs * targets).sum()
250         dice = (2. * intersection + self.smooth) / (inputs.sum() + targets.
    sum() + self.smooth)
251
```

```
        return 1 - dice

class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        ce_loss = F.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss

        if self.reduction == 'mean':
            return focal_loss.mean()
        else:
            return focal_loss.sum()

class TverskyLoss(nn.Module):
    def __init__(self, alpha=0.3, beta=0.7, smooth=1):
        super(TverskyLoss, self).__init__()
        self.alpha = alpha
        self.beta = beta
        self.smooth = smooth

    def forward(self, inputs, targets):

        probs = F.softmax(inputs, dim=1)
        inputs = probs[:, 1, :, :].contiguous().view(-1)
        targets = targets.contiguous().view(-1)

        TP = (inputs * targets).sum()
        FP = ((1 - targets) * inputs).sum()
        FN = (targets * (1 - inputs)).sum()

        tversky = (TP + self.smooth) / (TP + self.alpha * FP + self.beta * FN
     + self.smooth)

        return 1 - tversky

"""#### **5. Treinamento**"""

def calculate_metrics(pred_tensor, target_tensor, class_index=1):
    """
    Calcula IoU e Dice para uma classe específica (1 = Exsudatos).
    pred_tensor: (Batch, H, W) com 0s e 1s
```

```
    target_tensor: (Batch, H, W) com 0s e 1s
    """
    pred_flat = pred_tensor.view(-1)
    target_flat = target_tensor.view(-1)

    pred_class = (pred_flat == class_index)
    target_class = (target_flat == class_index)

    intersection = (pred_class & target_class).sum().float()

    pred_sum = pred_class.sum().float()
    target_sum = target_class.sum().float()

    union = pred_sum + target_sum - intersection

    smooth = 1e-6

    iou = (intersection + smooth) / (union + smooth)
    dice = (2. * intersection + smooth) / (pred_sum + target_sum + smooth)

    return iou.item(), dice.item()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Iniciando treino em: {device}")

# --- CONFIGURA  O DO EARLY STOPPING ---
PATIENCE = 7
patience_counter = 0

# Instanciando o modelo
model = MobileViTForSemanticSegmentation.from_pretrained(MODEL_CHECKPOINT,
    num_labels=2, ignore_mismatched_sizes=True)
model.to(device)

# Pesos para CrossEntropy
class_weights = torch.tensor([1.0, 10.0]).to(device)

# Instanciando as Loss Functions testadas
criterion_ce = nn.CrossEntropyLoss(weight=class_weights)
criterion_dice = DiceLoss()
criterion_focal = FocalLoss(alpha=0.8, gamma=2.0)
criterion_tversky = TverskyLoss(alpha=0.3, beta=0.7)

optimizer = AdamW(model.parameters(), lr=1e-4, weight_decay=1e-5)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS)

# Histórico para gráficos
```

```
history = {
    'train_loss': [], 'train_iou': [], 'train_dice': [],
    'val_loss': [], 'val_iou': [], 'val_dice': []
}

best_val_dice = 0.0

# Instantiate DataLoaders
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
    drop_last=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False)

# ==========================================
# 3. CICLO DE TREINO E VALIDA  O (LOOP)
# ==========================================

for epoch in range(EPOCHS):
    # --- FASE DE TREINO ---
    model.train()
    running_loss = 0.0
    running_iou = 0.0
    running_dice = 0.0

    train_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS} [Train]")

    for batch in train_bar:
        pixel_values = batch["pixel_values"].to(device)
        labels = batch["labels"].to(device)

        optimizer.zero_grad()

        # Forward Pass
        outputs = model(pixel_values=pixel_values)
        logits = outputs.logits

        # Redimensionar Logits
        logits = F.interpolate(
            logits,
            size=labels.shape[-2:],
            mode="bilinear",
            align_corners=False
        )

        # Calcular Loss Tversky
        loss = criterion_tversky(logits, labels)

        # Backward
```

```
390         loss.backward()
391         optimizer.step()
392 
393         # Métricas
394         with torch.no_grad():
395             probs = torch.softmax(logits, dim=1)[:, 1, :, :]
396             preds = (probs > 0.4).float()
397             iou, dice = calculate_metrics(preds, labels, class_index=1)
398 
399         running_loss += loss.item()
400         running_iou += iou
401         running_dice += dice
402 
403         train_bar.set_postfix(loss=loss.item(), dice=dice)
404 
405     # Médias de Treino
406     epoch_train_loss = running_loss / len(train_loader)
407     epoch_train_iou = running_iou / len(train_loader)
408     epoch_train_dice = running_dice / len(train_loader)
409 
410     history['train_loss'].append(epoch_train_loss)
411     history['train_dice'].append(epoch_train_dice)
412     history['train_iou'].append(epoch_train_iou)
413 
414     scheduler.step()
415     current_lr = scheduler.get_last_lr()[0]
416     print(f" poca  {epoch+1} finalizada. LR atual: {current_lr:.8f}")
417 
418     # --- FASE DE VALIDA  O ---
419     model.eval()
420     val_loss = 0.0
421     val_iou = 0.0
422     val_dice = 0.0
423 
424     count_valid_batches = 0
425 
426     with torch.no_grad():
427         for batch in tqdm(test_loader, desc=f"Epoch {epoch+1}/{EPOCHS} [Val]"
    ):
428             pixel_values = batch["pixel_values"].to(device)
429             labels = batch["labels"].to(device)
430 
431             outputs = model(pixel_values=pixel_values)
432             logits = outputs.logits
433 
434             logits = F.interpolate(
435                 logits,
```

```
436                 size=labels.shape[-2:],
437                 mode="bilinear",
438                 align_corners=False
439             )
440 
441             # Loss de validação (Monitoramento)
442             loss_v = criterion_tversky(logits, labels)
443             val_loss += loss_v.item()
444 
445             # Predição para métricas
446             probs = torch.softmax(logits, dim=1)[:, 1, :, :]
447             preds = (probs > 0.4).float()
448 
449             if labels.sum() > 0:
450                 iou, dice = calculate_metrics(preds, labels, class_index=1)
451                 val_iou += iou
452                 val_dice += dice
453                 count_valid_batches += 1
454 
455     # Médias de Validação
456     epoch_val_loss = val_loss / len(test_loader)
457 
458     # Se nenhum batch válido foi encontrado (improvável), evita divisão por
    zero
459     if count_valid_batches > 0:
460         epoch_val_iou = val_iou / count_valid_batches
461         epoch_val_dice = val_dice / count_valid_batches
462     else:
463         epoch_val_iou = 0.0
464         epoch_val_dice = 0.0
465 
466     history['val_loss'].append(epoch_val_loss)
467     history['val_dice'].append(epoch_val_dice)
468     history['val_iou'].append(epoch_val_iou)
469 
470     print(f"\nRESUMO EPOCH {epoch+1}:")
471     print(f"Train -> Loss: {epoch_train_loss:.4f} | Dice: {epoch_train_dice
    :.4f}")
472     print(f"Val   -> Loss: {epoch_val_loss:.4f}   | Dice: {epoch_val_dice:.4f
    }")
473 
474     # ==========================================
475     # L GICA DE SAVE + EARLY STOPPING
476     # ==========================================
477 
478     # Verifica se melhorou o Dice de Validação
479     if epoch_val_dice > best_val_dice:
```

```
        print(f"--> Melhora detectada ({best_val_dice:.4f} -> {epoch_val_dice
    :.4f}). Salvando modelo...")
        best_val_dice = epoch_val_dice
        patience_counter = 0 # Resetar paciência pois melhorou

        save_path = "/gdrive/MyDrive/Lumina/Models/Best_MobileViT_IDRID"
        model.save_pretrained(save_path)
        processor.save_pretrained(save_path)
    else:
        patience_counter += 1
        print(f"--> Sem melhora. Paciência: {patience_counter}/{PATIENCE}")

        # Condição de Parada
        if patience_counter >= PATIENCE:
            print("\n" + "="*50)
            print(f"EARLY STOPPING ATIVADO NA  POCA  {epoch+1}!")
            print(f"O modelo não melhora há {PATIENCE} épocas. Parando treino
    .")
            print("="*50)
            break # Encerra o loop 'for epoch'

    print("-" * 50)

"""#### **6. Avaliação**"""

epochs_range = range(1, len(history['train_loss']) + 1)

plt.figure(figsize=(12, 5))

# Gráfico de DICE
plt.subplot(1, 2, 1) # Agora 1 linha, 2 colunas, 1  gráfico
plt.plot(epochs_range, history['train_dice'], label='Train Dice')
plt.plot(epochs_range, history['val_dice'], label='Val Dice', linestyle='--')
plt.title('(a)')
plt.xlabel(' poca ')
plt.ylabel('Dice')
plt.legend()
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.yticks(np.arange(0,1.1,.1))
plt.xticks(np.arange(0,31,5))

# Gráfico de IoU
plt.subplot(1, 2, 2) # Agora 1 linha, 2 colunas, 2  gráfico
plt.plot(epochs_range, history['train_iou'], label='Train IoU')
plt.plot(epochs_range, history['val_iou'], label='Val IoU', linestyle='--')
plt.title('(b)')
plt.xlabel(' poca ')
```

```
plt.ylabel('IoU')
plt.legend()
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.yticks(np.arange(0,1.1,.1))
plt.xticks(np.arange(0,31,5))

plt.tight_layout() # Ajusta o layout para evitar sobreposição
plt.show()

print(f"--- Métricas do Modelo Salvo ---")
print(f"Dice Score (Validação): {final_best_dice:.4f}")
print(f"IoU (Validação): {final_best_iou:.4f}")
print(f"Tempo de Inferência por Imagem (média): {avg_inference_time_per_image
    :.4f} segundos")

plt.figure(figsize=(10, 6))
plt.plot(epochs_range, history['train_loss'], label='Train Loss')
plt.plot(epochs_range, history['val_loss'], label='Val Loss', linestyle='--')
plt.title('')
plt.xlabel(' poca ')
plt.ylabel('Loss')
plt.legend()
# grid mais fina e com linha tracejada
plt.grid(axis='y', linestyle='--', linewidth=0.5)
plt.yticks(np.arange(0,1.1,.1))
plt.xticks(np.arange(0,31,5))
epoch_no_improvement_start = 20
plt.axvline(x=epoch_no_improvement_start, color='r', linestyle=':', label='
    Fim da melhoria')
plt.legend()

plt.show()
```